

II

EXPLORANDO WIDGETS Y NAVEGACIÓN

En el presente capítulo, se explora el universo de los Stateless Widgets, acompañado de la presentación de una serie de widgets fundamentales que constituyen la base para el desarrollo de interfaces de usuario. Se examina la forma en que estos elementos facilitan la creación y organización visual de las aplicaciones de forma eficaz, brindando una introducción aplicada a la amplia gama de componentes de interfaz de usuario que ofrece Flutter.

Además, se aborda el concepto esencial de la navegación, mostrando cómo conectar diferentes pantallas y crear experiencias de usuario cohesivas y fluidas. Este conocimiento es fundamental para desarrollar aplicaciones que requieren múltiples vistas y flujos de interacción.

El capítulo concluye con un ejercicio práctico donde se aplicará lo aprendido, diseñando la interfaz de usuario para una aplicación de lista de planes. Este ejercicio plantea la integración de widgets y técnicas de navegación para crear una aplicación funcional y visualmente atractiva, consolidando el entendimiento de Flutter en la práctica.

2.1 Widgets

Los widgets son considerados los bloques fundamentales para la construcción de la interfaz de una aplicación en Flutter, ya que permiten definir su estructura y comportamiento específico. Al establecer una analogía con la construcción de una casa, los widgets se asemejan a los ladrillos y el cemento, es decir, a los materiales esenciales requeridos para edificar cada parte de un hogar. De la misma forma que se seleccionan distintos tipos de ladrillos y materiales para las paredes, el techo y el suelo, adaptándolos según la función y el diseño deseado para cada estancia, los widgets permiten construir y personalizar la interfaz de una aplicación en Flutter. Cada widget tiene un papel en la estructura general y en el comportamiento específico de la aplicación, de manera similar a cómo cada material contribuye al diseño y funcionalidad de cada espacio en una casa.

Los widgets se organizan jerárquicamente en una estructura de árbol, donde un widget puede albergar otros widgets como hijos, esta organización jerárquica determina tanto la estructura como el aspecto visual de la interfaz de usuario.

En Flutter, existen principalmente tres categorías de widgets: los Stateless Widgets, que son widgets que no tienen estado; los Stateful Widgets, con widgets que tienen un estado; y los Inherited Widgets, que son widgets heredados. Por su simplicidad, resulta conveniente comenzar explorando los Stateless Widgets, dejando los Stateful Widgets e Inherited Widgets para profundizar en el siguiente capítulo.

2.1.1 Stateless Widgets (Widgets sin estado)

Una interfaz de usuario (UI por sus términos en inglés: User Interface) típica se compone de numerosos widgets y algunos de estos mantendrán sus propiedades inalteradas tras su creación, estos son los widgets sin estado, caracterizados por no experimentar cambios por sí mismos a través de acciones o comportamientos internos. En cambio, cualquier modificación en ellos proviene de eventos que ocurren en los widgets padres situados en el árbol de widgets. Por ende, se puede afirmar que los widgets sin estado delegan el control de su construcción a un widget padre. El siguiente diagrama ilustra cómo se ven los widgets sin estado:

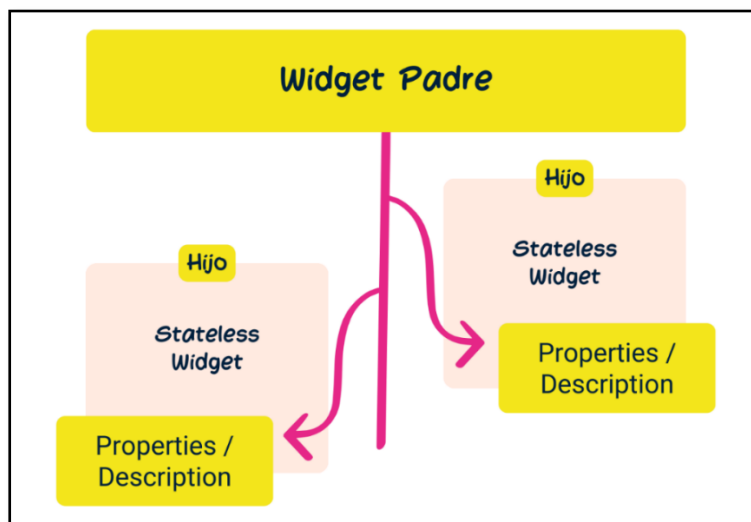


Figura II.1. Stateless Widgets adaptado de *Flutter for beginners*, por Bailey & Biessek, p. (2021, p. 115)

En la figura anterior, el widget padre instancia a los widgets hijo sin estado y pasa un conjunto de propiedades durante la instanciación. Los widgets hijo solo pueden recibir estas propiedades del widget padre y no las cambiarán por sí mismo. En términos de código, esto significa que los widgets sin estado solo tienen propiedades finales definidas durante la construcción y estas propiedades solo pueden ser cambiadas a través de la actualización del widget padre, con los cambios extendiéndose entonces a los widgets hijos.

El siguiente es el código básico que define a un widget sin estado:

```
class MyApp extends StatelessWidget {  
  const MyApp({super.key});  
  @override  
  Widget build(BuildContext context) {  
    ...  
  }  
}
```

Como se puede ver, la clase *MyApp* extiende *StatelessWidget*, por lo que se considera como un widget sin estado. Dentro la clase se sobrescribe el método *build*, es necesario hacerlo para describir cómo debe aparecer el widget en la pantalla, esto lo hace construyendo un subárbol de widgets debajo de él.

BuildContext es un argumento proporcionado al método *build* como una forma útil de interactuar con el árbol de widgets, lo que permite acceder a información ancestral que ayuda a describir el widget que se está construyendo. Por ejemplo, los datos del tema definidos en este widget pueden ser accedidos por todos los widgets hijos para asegurar que haya una apariencia y sensación consistentes en toda la aplicación.

2.1.2 La propiedad *key*

Según Zaccagnino, p. (2020, p. 74) “La *Key* se utiliza para identificar de manera única a un widget y es utilizada por el framework para actualizar los elementos que realmente necesitan ser actualizados.” [Traducción propia]. Por esa razón, la propiedad *key* es considerada fundamental para el control de cómo se identifican y mantienen los widgets dentro del árbol de widgets. Durante el desarrollo de aplicaciones, es frecuente encontrarse con situaciones en las cuales se requiere insertar, remover o reordenar elementos dentro de la interfaz de usuario, en estos contextos, la propiedad *key* adquiere relevancia para mantener el estado y la identidad de los widgets.

A continuación, se presentan algunos puntos clave sobre la finalidad de la propiedad *key*:

- **Identificación única:** Las *keys* proporcionan una identificación única para los widgets dentro de la misma jerarquía, lo que resulta útil en situaciones donde múltiples elementos del mismo tipo pueden confundirse entre sí.
- **Preservación del estado:** En listas dinámicas o estructuras donde los widgets se generan de manera dinámica, la propiedad *key* asiste a Flutter en determinar qué elementos han cambiado, cuáles necesitan ser recreados y cuáles pueden ser reutilizados. Esto juega un papel importante en la preservación del estado de los widgets, por ejemplo, la posición de desplazamiento en una lista.

- **Optimización del rendimiento:** Al facilitar la identificación eficiente de los widgets que requieren ser reconstruidos por parte del framework, las keys contribuyen a optimizar el rendimiento de la aplicación evitando la reconstrucción innecesaria de widgets que no han experimentado cambios.
- **Gestión de elementos en colecciones:** En colecciones donde los elementos pueden cambiar de posición, como en una lista que puede ser reordenada por el usuario, las keys aseguran que cada elemento conserve adecuadamente su estado incluso después de ser movido.

En conclusión, la propiedad *key* se destaca como un elemento esencial en Flutter para asegurar la consistencia y el rendimiento de la interfaz de usuario, especialmente en escenarios dinámicos y en listas que experimentan cambios a lo largo del tiempo, su implementación adecuada es significativa para el desarrollo de aplicaciones fluidas y eficientes.

2.2 Catálogo de Widgets básicos

Los widgets se constituyen en un elemento fundamental en la filosofía de Flutter, facilitando la creación de interfaces de usuario flexibles y personalizadas de manera eficiente. La composición de widgets permite la construcción de interfaces de usuario complejas que pueden ser reutilizadas en distintas partes de una aplicación. Además, Flutter ofrece una amplia gama de widgets predefinidos que simplifican la creación de aplicaciones con una apariencia y comportamiento consistentes.

En esta sección, se revisarán los widgets que proporciona Flutter, los cuales probablemente son de los más empleados al iniciar el viaje con esta tecnología. No obstante, es importante señalar que existe una extensa variedad de widgets adicionales disponibles para ser explorados en el catálogo de widgets¹¹ del sitio web oficial de Flutter.

2.2.1 El Widget MaterialApp

Payne, p. (2019, p. 100) destaca la función esencial del widget *MaterialApp* en el desarrollo de aplicaciones con Flutter, señalando su papel en la creación del marco externo de la aplicación. A pesar de su importancia fundamental, el *MaterialApp* permanece invisible para el usuario, ya que su función es envolver la aplicación por completo sin mostrar ninguna de sus partes de manera directa. Este widget no solo proporciona una identidad a la aplicación al permitir asignarle un título para su identificación cuando se minimiza, sino que también establece el tema predeterminado que definirá aspectos visuales clave como fuentes, tamaños y colores.

El widget *MaterialApp* es un widget que se emplea para envolver varios widgets que son comúnmente requeridos para aplicaciones que siguen las directrices de diseño de Material Design, actúa como un widget de nivel superior que se utiliza al inicio de la aplicación para configurar y definir la estructura básica de la aplicación, incluyendo la navegación, los temas y la localización, entre otras cosas. Aquí están algunos de los aspectos más importantes y usos del widget *MaterialApp*:

¹¹ <https://docs.flutter.dev/ui/widgets>

- **Estructura de la Aplicación:** *MaterialApp* sirve como el punto de entrada para la mayoría de las aplicaciones que implementan Material Design, esta se coloca en la raíz del árbol de widgets de la aplicación.
- **Navegación:** Gestiona la navegación y las rutas en la aplicación, *MaterialApp* utiliza el *Navigator* para manejar el stack de rutas de la aplicación, se pueden definir rutas nombradas dentro de *MaterialApp* para facilitar la navegación entre pantallas. Todo ello se estudiará con detalle en la siguiente sección.
- **Tema:** Permite definir un tema global para la aplicación a través del parámetro *theme*, esto incluye colores, tipografías y estilos de widgets que se aplicarán a lo largo de toda la aplicación, asegurando la consistencia visual.
- **Localización:** *MaterialApp* facilita la localización de aplicaciones, permitiendo que se adapten a diferentes idiomas y regiones, soporta tanto el texto de izquierda a derecha (LTR) como de derecha a izquierda (RTL).
- **Pantalla de Inicio (Splash Screen):** Puede mostrar una pantalla de inicio mientras la aplicación está cargando, configurando el parámetro *splashScreen*.
- **Widgets de Material Design:** Al utilizar *MaterialApp*, se garantiza que la aplicación pueda utilizar plenamente los widgets de Material Design, como *Scaffold*, *AppBar*, entre otros, que siguen las directrices de Material Design.

Las siguientes son algunas de las propiedades más importantes de *MaterialApp*:

- *home*: Define el widget que se mostrará cuando la aplicación se ejecute y no haya ninguna ruta específica siendo accedida.
- *routes*: Es un mapa de rutas nombradas que facilita la navegación basada en nombres a través de la aplicación.
- *onGenerateRoute*: Se utiliza para manejar rutas dinámicas o generadas en tiempo de ejecución.
- *Locale*: Define la configuración regional de la aplicación.
- *supportedLocales*: Una lista de locales que la aplicación soporta.
- *localizationsDelegates*: Define los delegados de localización responsables de cargar y construir las localizaciones específicas de la aplicación.

El siguiente es un ejemplo básico para el uso de *MaterialApp*:

```
class MyApp extends StatelessWidget {  
  const MyApp({super.key});  
  @override  
  Widget build(BuildContext context) {  
    return const MaterialApp(  
      debugShowCheckedModeBanner: false,
```

```
        home: MyScreen(),  
      );  
    }  
  }
```

El código anterior define una clase *MyApp* que extiende *StatelessWidget*, indicando que *MyApp* es un widget sin estado, es decir, no maneja cambios internos de estado a lo largo del tiempo. Dentro de *MyApp*, el constructor permite pasar opcionalmente una clave al widget padre a través de *super.key*. En el método *build*, se construye el árbol de widgets de *MyApp*, este retorna un widget *MaterialApp*, configurado para no mostrar el banner de depuración que aparece en la esquina superior derecha (*debugShowCheckedModeBanner: false*), lo que es útil para crear una apariencia más limpia durante la presentación o producción de la aplicación. El widget *MaterialApp* define a *MyScreen* como el widget de inicio (*home*) de la aplicación, lo que significa que *MyScreen* es el primer widget que se muestra cuando la aplicación se ejecuta, sirviendo como la pantalla principal de la aplicación.

Más detalles de empleo de *MaterialApp*¹² se pueden encontrar en la documentación citada a pie de página.

2.2.2 El Widget Scaffold

Según Katz et al., p. (2021, p. 90), “el widget Scaffold implementa todas tus necesidades básicas de estructura de diseño visual” [Traducción propia]. Estas necesidades básicas, de manera precisa, se refieren a que el Scaffold facilita la creación de una estructura básica de interfaz de usuario, ofreciendo un marco de trabajo con elementos predefinidos y espacios para implementar componentes, como barras de aplicación, cajones de navegación, barras de navegación inferiores y cuerpos flotantes de acción (FABs), entre otros. Esos elementos se pueden apreciar en la siguiente imagen:

¹² <https://api.flutter.dev/flutter/material/MaterialApp-class.html>

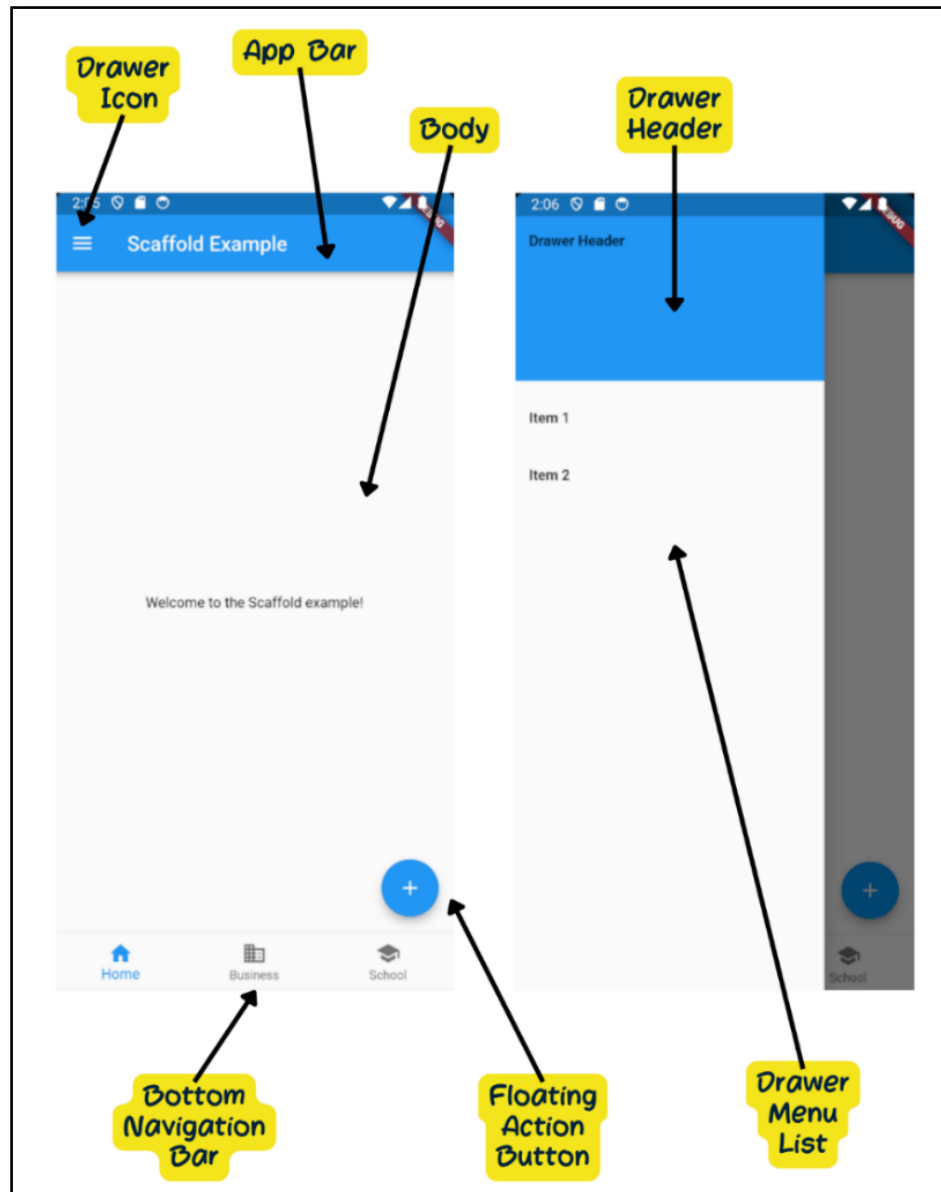


Figura II.2. Elementos de Scaffold

El código empleado para generar el anterior par de vistas es el siguiente, cabe señalar que se trata de la clase *MyScreen* que es hijo del widget *MaterialApp* ejemplificado en el anterior punto.

```
class MyScreen extends StatelessWidget {
  const MyScreen({super.key});
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Scaffold Example'),),
      body: const Center(child: Text('Welcome to the Scaffold example!'),),
      floatingActionButton: const FloatingActionButton(
```

```

        onPressed: null, tooltip: 'Add', child: Icon(Icons.add),
      ),
      drawer: Drawer(
        child: ListView(
          padding: EdgeInsets.zero,
          children: [
            const DrawerHeader(
              decoration: BoxDecoration(color: Colors.blue,),
              child: Text('Drawer Header'),
            ),
            ListTile(
              title: const Text('Item 1'),
              onTap: () {Navigator.pop(context);},
            ),
            ListTile(
              title: const Text('Item 2'),
              onTap: () {Navigator.pop(context);},
            ),
          ],
        ),
      ),
      bottomNavigationBar: BottomNavigationBar(
        items: const <BottomNavigationBarItem>[
          BottomNavigationBarItem(
            icon: Icon(Icons.home), label: 'Home',
          ),
          BottomNavigationBarItem(
            icon: Icon(Icons.business), label: 'Business',
          ),
          BottomNavigationBarItem(
            icon: Icon(Icons.school), label: 'School',
          ),
        ],
      ),
    );
  }
}

```

Como se mencionó *Scaffold* proporciona la estructura básica de la página, incluyendo elementos como la barra de aplicaciones (*AppBar*), el cuerpo principal (*body*), un panel inferior (*bottomNavigationBar*), un botón de acción flotante (*FloatingActionButton*) y más. A continuación, se observa en detalle lo que representa cada uno de los elementos presentes en el código anterior.

- *AppBar*: El *AppBar* es uno de los elementos más utilizados en *Scaffold*, proporciona una barra en la parte superior de la pantalla que suele contener el título de la página y acciones como botones o menús.
- *body*: El *body* es el área principal donde se coloca el contenido principal de la pantalla, puede contener cualquier tipo de widget y es donde se desarrolla la mayor parte de la interfaz de usuario.
- *floatingActionButton*: El *floatingActionButton* es un botón circular que se coloca generalmente en la esquina inferior derecha del *Scaffold*, se utiliza para acciones primarias en la pantalla, como agregar, editar o realizar una tarea importante.
- *drawer*: El *Scaffold* permite agregar un menú lateral deslizable, conocido como *drawer*, que se utiliza para navegación o mostrar opciones adicionales.
- *bottomNavigationBar*: Proporciona una barra de navegación en la parte inferior, útil para cambiar entre diferentes vistas o secciones de la aplicación.

También se tiene otros elementos, que, si bien no se tomaron en cuenta en el anterior ejemplo, pueden resultar útiles, estas son: *SnackBar*, *BottomSheet* y *Dialogs*.

El *Scaffold* facilita la implementación de elementos temporales como *SnackBar* (mensaje breve en la parte inferior de la pantalla), *BottomSheet* (un panel que se desliza desde la parte inferior) y diálogos modales.

Por otro lado, es importante indicar que el *Scaffold* maneja el estado de estos elementos y la interacción entre ellos, como mostrar u ocultar un *SnackBar* cuando se presiona un botón.

Además, una gran cualidad del *Scaffold* es que es altamente personalizable, ya que permite cambiar el tema, el color de fondo, la tipografía y otros aspectos visuales para que coincidan con el diseño de la aplicación que estamos desarrollando.

En resumen, el *Scaffold* es un componente esencial en el desarrollo de aplicaciones, ya que proporciona una estructura estándar para la mayoría de las interfaces de usuario lo que facilita la implementación de elementos comunes de diseño de Material Design. Su flexibilidad y facilidad de uso lo convierten en una herramienta indispensable.

Como en toda tecnología, es recomendable recurrir a la documentación oficial para obtener detalles del widget *Scaffold*¹³, esto porque en la documentación se puede encontrar un nivel de detalles mayor al que se intenta presentar en este libro.

2.2.3 El Widget Text

El widget *Text* es esencial para cualquier aplicación móvil, su propósito principal es mostrar una cadena de texto en la interfaz de usuario. El siguiente código muestra un ejemplo de su empleo:

¹³ <https://api.flutter.dev/flutter/material/Scaffold-class.html>

```
Text(  
  "Hola Mundo",  
  style: TextStyle(color: Colors.red, fontSize: 14),  
  textAlign: TextAlign.center,  
),
```

Algunos aspectos clave sobre el widget `Text` se detallan a continuación:

- **Visualización de texto:** El widget `Text` permite mostrar texto, puede ser tan simple como un texto estático o tan complejo como un texto dinámicamente generado basado en la interacción del usuario o en datos en tiempo real.
- **Estilización:** El widget `Text` ofrece una amplia gama de opciones de estilización, es posible cambiar el estilo del texto, incluyendo su fuente, tamaño, color, grosor, espaciado entre letras y más. Esto se logra a través del parámetro `style` del widget, que toma un objeto `TextStyle`.
- **Alineación y dirección del texto:** Ofrece control sobre la alineación (como centrado, alineado a la izquierda, etc.) y la dirección del texto (LTR para idiomas de izquierda a derecha, RTL para idiomas de derecha a izquierda) lo cual es importante para la internacionalización.
- **Text Wrapping y Overflow:** `Text` gestiona automáticamente el ajuste de líneas y el desbordamiento de texto, es posible definir cómo se debe comportar el texto cuando es demasiado largo para su contenedor, como truncarlo o mostrar puntos suspensivos.
- **Accesibilidad:** Flutter ha puesto un gran énfasis en la accesibilidad y el widget `Text` no es una excepción, `Text` asegura que el texto sea legible y accesible, con soporte para características de accesibilidad como el tamaño de fuente dinámico.
- **Interacción con otros widgets:** `Text` se utiliza a menudo en combinación con otros widgets, como dentro de un `Column` o `Row` o junto con botones e inputs. También es común envolverlo en un `GestureDetector` para capturar toques o gestos.
- **Localización e internacionalización:** Es compatible con la localización, permitiendo mostrar diferentes textos para diferentes idiomas y regiones, lo cual es fundamental para aplicaciones globalizadas.

En resumen, el widget `Text` es un componente imprescindible en Flutter, utilizado en casi todas las aplicaciones para mostrar información en forma de texto, su versatilidad y las amplias opciones de personalización lo hacen adecuado para una gran variedad de necesidades de visualización de texto.

Para ver todas las propiedades disponibles del widget `Text`, se debe consultar la documentación oficial en la sección relacionada al widget `Text`¹⁴.

¹⁴ <https://api.flutter.dev/flutter/widgets/Text-class.html>

2.2.4 El Widget Image

El widget *Image* en Flutter es un componente esencial para mostrar imágenes en la interfaz de usuario de una aplicación, este widget proporciona una forma flexible y potente de trabajar con recursos gráficos.

El siguiente código ejemplifica como visualizar una imagen desde una URL.

```
Image.network(  
  'https://cdn.pixabay.com/photo/2023/12/14/20/24/christmas-balls-8449616_1280.jpg',  
  width: 200,  
  height: 200,  
  fit: BoxFit.cover,  
),
```

A continuación, se describen los aspectos más destacados del widget *Image*:

- **Carga de imágenes:** Flutter permite cargar imágenes de diversas fuentes, incluyendo activos locales (archivos dentro del proyecto), redes (imágenes de Internet), archivos en disco y memoria. Esto se logra utilizando diferentes constructores como *Image.asset*, *Image.network*, *Image.file* y *Image.memory*.
- **Adaptabilidad y escalado:** El widget *Image* maneja automáticamente la adaptabilidad de las imágenes a diferentes tamaños de pantalla y resoluciones de dispositivos. Ofrece varias opciones para el escalado de imágenes, como ajustarlas para llenar el espacio disponible (*BoxFit.fill*), mantener la relación de aspecto (*BoxFit.contain*), cubrir el espacio disponible manteniendo la relación de aspecto (*BoxFit.cover*), etc.
- **Manipulación de la imagen:** Se pueden aplicar efectos como recorte, rotación o mezclado de colores directamente a través del widget *Image*, sin necesidad de manipulación previa de la imagen.
- **Gestión del cache:** Flutter gestiona de manera eficiente la caché de imágenes, especialmente para las imágenes obtenidas de la red, esto mejora el rendimiento al reducir la cantidad de veces que se deben cargar las imágenes desde Internet.
- **Placeholder y manejo de errores:** Con *Image.network*, puedes especificar un widget para mostrar mientras la imagen se está cargando (*placeholder*) y otro para mostrar en caso de que la carga falle (*errorBuilder*).
- **Animaciones y GIFs:** El widget *Image* soporta la visualización de imágenes animadas, como GIFs, lo cual permite añadir dinamismo a la interfaz de usuario.
- **Accesibilidad:** Al igual que otros widgets en Flutter, *Image* se puede configurar para ser más accesible, por ejemplo, utilizando la propiedad *semanticLabel* para describir la imagen a usuarios que utilizan lectores de pantalla.

- **Decoración:** Además de usarse directamente, las imágenes en Flutter también pueden ser utilizadas como parte de la decoración de otros widgets, por ejemplo, como fondo de un Container mediante *BoxDecoration*.

En resumen, el widget *Image* en Flutter es una herramienta versátil para la visualización de imágenes, ofrece una amplia gama de funcionalidades para cubrir casi cualquier necesidad relacionada con la manipulación y presentación de imágenes en aplicaciones móviles.

Sin embargo, si se presenta alguna necesidad especial, es recomendable ver todas las propiedades disponibles del widget *Image*, para ello, consulta la página oficial de la documentación del widget *Image*¹⁵.

2.2.5 Los Widgets para botones

El manejo de botones en Flutter es muy importante para la interacción con el usuario en cualquier aplicación móvil, Flutter proporciona varios widgets de botón, cada uno con sus características y usos específicos.

A continuación, se detallan los distintos tipos de botones y su manejo.

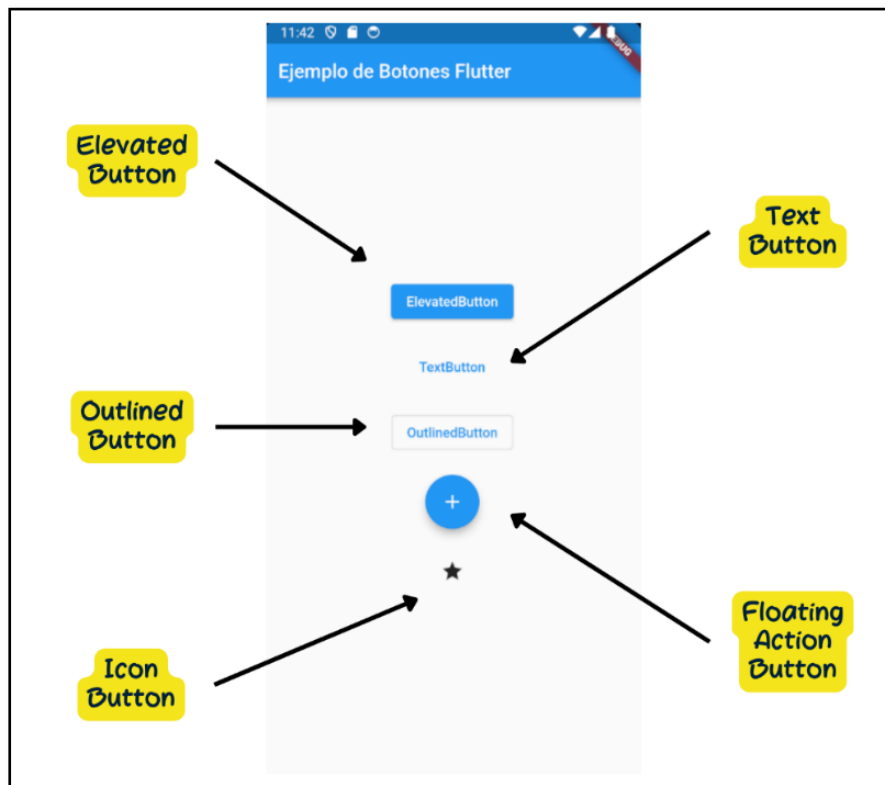


Figura II.3. Tipos de Botones

¹⁵ <https://api.flutter.dev/flutter/widgets/Image-class.html>

ElevatedButton¹⁶

Es un widget que representa un botón con una superficie elevada, este widget es una parte común de la interfaz de usuario en muchas aplicaciones y se utiliza para acciones que el usuario puede realizar al tocar el botón.

El siguiente, es un ejemplo de cómo emplear este widget.

```
ElevatedButton(  
  onPressed: null,  
  child: Text('ElevatedButton'),  
),
```

TextButton¹⁷

TextButton es otro widget que se utiliza para crear botones, pero a diferencia de *ElevatedButton*, es un botón de texto plano, es decir, no tiene una superficie elevada o efecto de elevación visual, esto lo hace una opción adecuada cuando se desea un botón de aspecto más simple y minimalista en la interfaz de usuario.

A continuación, puede ver el código para emplear un *TextButton*

```
TextButton(  
  onPressed: null,  
  child: Text('TextButton'),  
),
```

OutlinedButton¹⁸

También se utiliza para crear botones y se diferencia de *ElevatedButton* y *TextButton* en su apariencia visual, a diferencia de *ElevatedButton*, que tiene una superficie con efecto de elevación y de *TextButton* que es un botón de texto plano, *OutlinedButton* tiene un borde visible y un fondo transparente, esto le da un aspecto distintivo a los botones *OutlinedButton*, ya que parecen botones con bordes dibujados, pero sin un fondo sólido.

A continuación, puede observar un ejemplo de su sintaxis:

```
OutlinedButton(  
  onPressed: null,  
  child: Text('OutlinedButton'),  
),
```

¹⁶ <https://api.flutter.dev/flutter/material/ElevatedButton-class.html>

¹⁷ <https://api.flutter.dev/flutter/material/TextButton-class.html>

¹⁸ <https://api.flutter.dev/flutter/material/OutlinedButton-class.html>

IconButton¹⁹

Es un widget que se utiliza para crear botones que representan iconos, es una forma conveniente de incorporar iconos interactivos en la interfaz de usuario y es útil para acciones que el usuario puede realizar al tocar un icono específico.

El siguiente código ejemplifica su escritura:

```
IconButton(  
  icon: Icon(Icons.add_alert),  
  onPressed: null,  
),
```

FloatingActionButton²⁰

Es un widget que se utiliza para crear un botón de acción flotante en una interfaz de usuario. El botón de acción flotante es un elemento visual comúnmente utilizado en aplicaciones móviles para representar una acción principal o de alto nivel que el usuario puede realizar en la pantalla actual.

```
floatingActionButton: const FloatingActionButton(  
  onPressed: null,  
  child: Icon(Icons.add),  
),
```

Al tratarse de botones, todos tienen propiedades comunes, estas se describen a continuación:

- **onPressed:** Es un callback que se llama cuando el botón es presionado, si es *null*, el botón se deshabilita. Los botones pueden cambiar de estado (habilitado/deshabilitado) basado en la lógica de la aplicación, esto se maneja típicamente cambiando la propiedad *onPressed*.
- **child:** El contenido del botón, que puede ser un texto, icono o cualquier widget.
- **style:** Define el estilo del botón, como el color de fondo, forma, tamaño, etc. Los botones pueden ser altamente personalizados, es decir, es posible cambiar su forma, color, comportamiento al presionar, sombras, etc.

En la práctica, el manejo de botones en Flutter implica no solo colocar un botón en la interfaz de usuario, sino también gestionar su estado, estilo y la forma en que interactúa con el resto de la aplicación, esto es clave para crear una experiencia de usuario intuitiva y agradable.

¹⁹ <https://api.flutter.dev/flutter/material/IconButton-class.html>

²⁰ <https://api.flutter.dev/flutter/material/FloatingActionButton-class.html>

2.2.6 Los Widgets para layouts

El layout se refiere a cómo se organiza y se presentan los elementos visuales en la interfaz de usuario de una aplicación, es decir, determinan la disposición y la estructura de los elementos. Los layouts son fundamentales para la creación y estructuración de la interfaz de usuario (UI).

En Flutter se utiliza varios widgets de tipo layout para controlar cómo se posicionan, se dimensionan y se organizan los widgets en la pantalla, los más comunes son: *Column*, *Row*, *Stack*, *Container*, *GridView* y *ListView*.

Column²¹

El widget *Column* se emplea para organizar widgets hijos de manera vertical, uno debajo del otro, es útil cuando desea apilar widgets verticalmente, como listas de elementos, botones o cualquier otro contenido que deba colocarse en una columna vertical.

El siguiente ejemplo ilustra su codificación.

```
Column(  
  mainAxisAlignment: MainAxisAlignment.center,  
  children: [  
    Text('Widget 1'),  
    Text('Widget 2'),  
    Text('Widget 3'),  
    Text('Widget 4'),  
    Text('Widget 5'),  
  ],  
)
```

Column es muy versátil y ofrece diversas propiedades para controlar el diseño y la presentación de sus widgets hijos, por ejemplo, se puede especificar la lista de widgets que se mostraran en el *Column* mediante la propiedad *children*, esta lista debe estar dentro de corchetes, también, es posible especificar la alineación de los widgets hijos dentro de la columna, para ello, se emplea la propiedad *mainAxisAlignment*, esta es una propiedad que determina cómo los hijos del *Column* se deben alinear a lo largo del eje principal, en el caso de *Column*, el eje principal es vertical, ya que *Column* dispone sus hijos de arriba hacia abajo.

La propiedad *mainAxisAlignment* acepta valores del tipo *MainAxisAlignment*, que incluyen:

- *MainAxisAlignment.start*: Alinea los hijos al inicio del eje vertical, en un *Column* esto significa alinearlos en la parte superior.
- *MainAxisAlignment.center*: Centra los hijos en el eje vertical.

²¹ <https://api.flutter.dev/flutter/widgets/Column-class.html>

- *MainAxisAlignment.end*: Alinea los hijos al final del eje vertical, en un *Column*, los hijos se alinean en la parte inferior.
- *MainAxisAlignment.spaceBetween*: Distribuye los hijos de manera uniforme en el eje vertical, dejando un espacio igual entre cada hijo.
- *MainAxisAlignment.spaceAround*: Similar a *spaceBetween*, pero el espacio alrededor de cada hijo es también la mitad del espacio entre los hijos.
- *MainAxisAlignment.spaceEvenly*: Distribuye los hijos de manera uniforme en el eje vertical, con un espacio igual entre los hijos y también igual al espacio del inicio y el final del eje.

La siguiente imagen muestra los distintos valores que puede tomar *mainAxisAlignment* y sus efectos en el widget *Column*.



Figura II.4. Valores y sus efectos para *mainAxisAlignment* en *Column*

Como se puede apreciar en la imagen, los widgets hijos se ajustan automáticamente para ocupar el espacio disponible verticalmente.

Por otro lado, es posible hacer una alineación en el eje secundario, que en el caso de *Column* sería el eje horizontal, para alinear los widgets hijos según el eje secundario se utiliza la propiedad *crossAxisAlignment*.

La propiedad *crossAxisAlignment* acepta valores del tipo *CrossAxisAlignment*, que incluyen:

- *CrossAxisAlignment.start*: Alinea los hijos al inicio del eje horizontal, en un contexto de izquierda a derecha, esto significa alinear los hijos a la izquierda.
- *CrossAxisAlignment.end*: Alinea los hijos al final del eje horizontal, en un contexto de izquierda a derecha, los hijos se alinearán a la derecha.
- *CrossAxisAlignment.center*: Centra los hijos en el eje horizontal.
- *CrossAxisAlignment.stretch*: Estira los hijos para que llenen el espacio horizontal disponible, esto ignora el ancho definido por los hijos y los expande para ocupar todo el espacio disponible a lo ancho.
- *CrossAxisAlignment.baseline*: Alinea los hijos según la línea base del texto, esta opción es útil cuando se tienen widgets de texto de diferentes tamaños y se quiere alinear el texto de manera consistente. Requiere que el *textBaseline* esté configurado.

Usar *crossAxisAlignment* es esencial para controlar cómo los elementos se distribuyen y alinean horizontalmente dentro de un *Column*, a continuación, puedes observar el resultado de la aplicación de cada uno de los posibles valores.

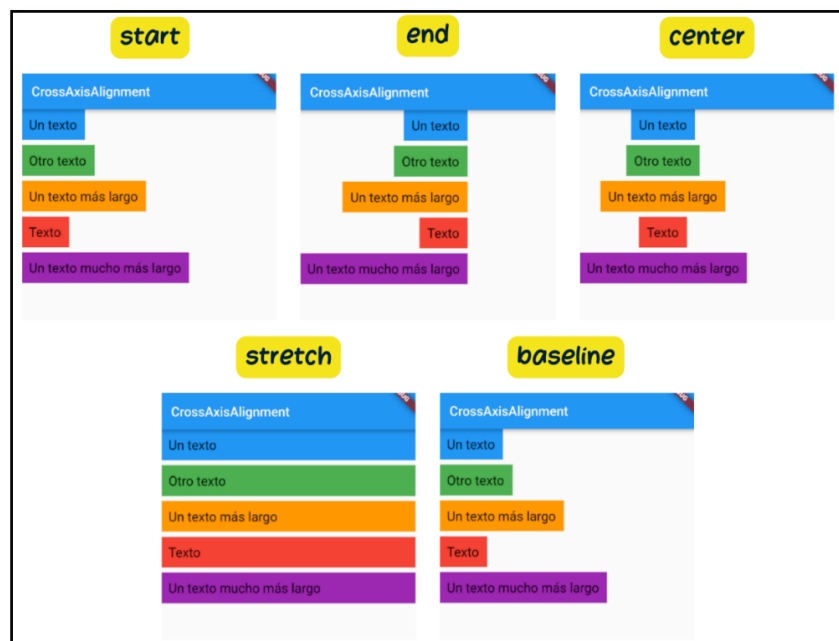


Figura II.5. Valores y sus efectos para *crossAxisAlignment* en *Column*

El tamaño de la columna a lo largo de su eje principal puede ser afectado con la propiedad *mainAxisSize*, este puede ser *MainAxisSize.min* lo que significa que la columna es tan grande como sus hijos lo necesiten o *MainAxisSize.max*, lo que significa que la columna se expandirá para llenar el espacio disponible en su eje principal.

Para definir el orden en el que se dibujarán los hijos verticalmente se puede utilizar la propiedad *verticalDirection*, es así que, *VerticalDirection.down* indica presentarlos de arriba a abajo o *VerticalDirection.up* los dibuja de abajo hacia arriba.

Row²²

El widget *Row* dispone a sus hijos en una línea horizontal, ofrece varias propiedades que permiten controlar cómo se distribuyen y alinean estos hijos, así como el propio tamaño y su comportamiento. El siguiente es un ejemplo que permite ver cómo se puede codificar:

```
Row(  
  mainAxisAlignment: MainAxisAlignment.center,  
  children: [  
    Icon(Icons.star),  
    Text('Valoración: 4.5'),  
  ],  
)
```

El manejo de las propiedades es similar al del widget *Column*, simplemente habrá que considerar que, en este caso el eje principal es el horizontal y el eje secundario es el vertical. En ese sentido, al igual que en *Column*, *children* sirve para indicar la lista de widgets que se colocarán horizontalmente en el *Row*.

La propiedad *mainAxisAlignment* controla como los hijos se alinean a lo largo del eje principal (horizontal) del *Row*. El resultado se observa en la siguiente imagen:

²² <https://api.flutter.dev/flutter/widgets/Row-class.html>

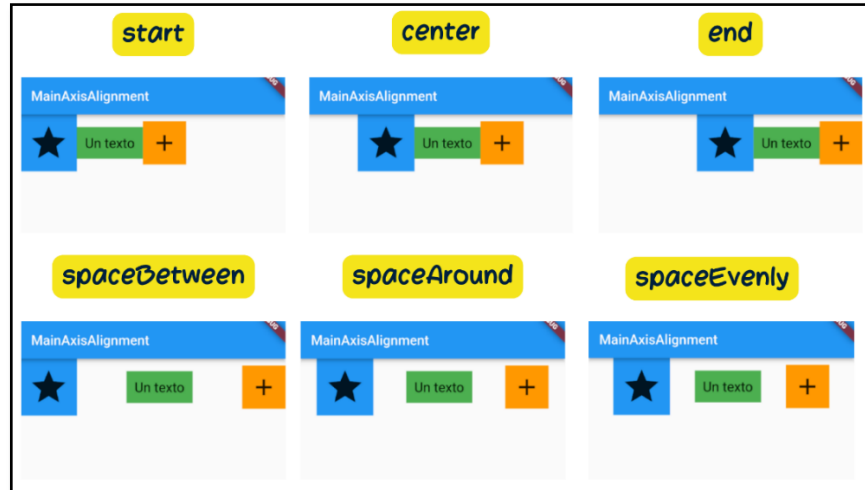


Figura II.6. Valores para `mainAxisAlignment` en `Row`

La propiedad `crossAxisAlignment` determina cómo los hijos se alinean a lo largo del eje secundario (vertical) del `Row`. Los valores posibles son:

- `CrossAxisAlignment.start`
- `CrossAxisAlignment.end`
- `CrossAxisAlignment.center`
- `CrossAxisAlignment.stretch`
- `CrossAxisAlignment.baseline`.

Los efectos generados por cada valor posible se pueden apreciar en la imagen que continúa:

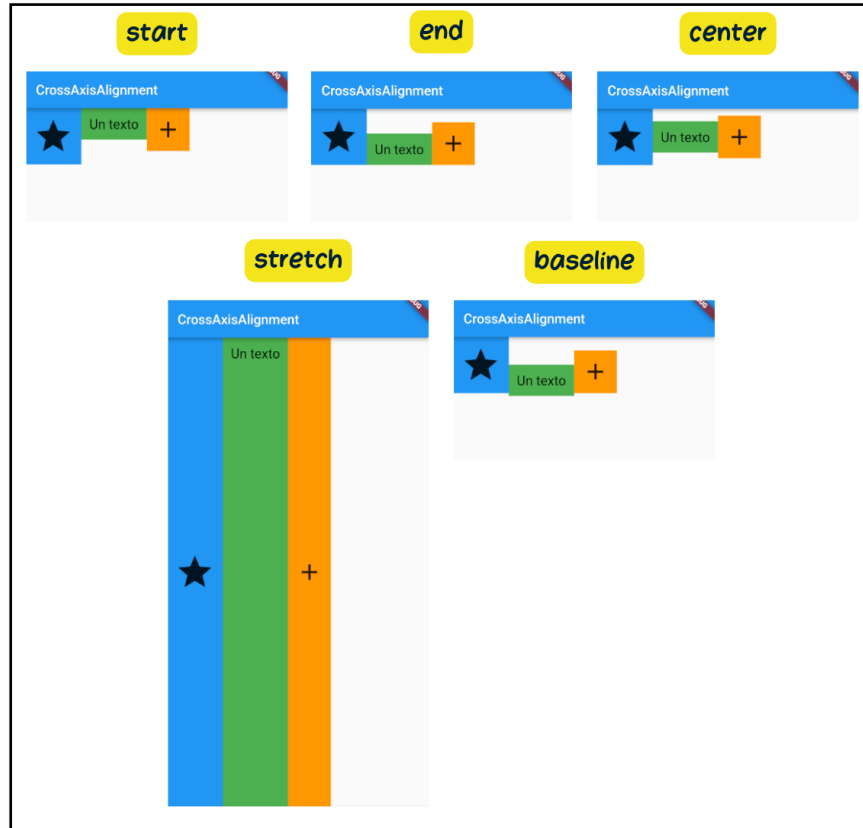


Figura II.7. Valores para `crossAxisAlignment` en `Row`

El tamaño del `Row` a lo largo de su eje principal se afecta con la propiedad `mainAxisSize`, que puede ser `MainAxisSize.min`, lo que significa que el `Row` es tan grande como sus hijos lo necesiten o `MainAxisSize.max`, lo que significa que el `Row` se expandirá para llenar el espacio disponible en su eje principal.

Stack²³

Es un widget que se utiliza para apilar widgets uno encima del otro en una superposición, se puede pensar en un `Stack` como una pila de elementos, donde los elementos se apilan verticalmente, donde es posible controlar la posición y el orden de apilamiento de cada widget dentro de la pila. El siguiente código es una referencia de cómo utilizarlo:

```
Stack(
  alignment: Alignment.center,
  children: [
    Container(
      width: 200,
      height: 200,
```

²³ <https://api.flutter.dev/flutter/widgets/Stack-class.html>

```
        color: Colors.blue,  
      ),  
      Positioned(  
        top: 50,  
        left: 50,  
        child: Text(  
          'Widget apilado',  
          style: TextStyle(color: Colors.white),  
        ),  
      ),  
    ],  
  )
```

Los widgets hijos de *Stack* se apilan uno encima del otro en orden, desde el primero en la parte inferior hasta el último en la parte superior, esto significa que el primer hijo agregado estará en la parte inferior de la pila y el último en la parte superior.

Puedes controlar la posición de cada widget hijo utilizando propiedades como *alignment* y *positioned*. La propiedad *alignment* te permite alinear todos los widgets hijos dentro de la pila en relación con el área de la pila. Por otro lado, la propiedad *positioned* te permite definir la posición exacta y el tamaño de un widget hijo de forma independiente.

El tamaño de la pila (*Stack*) se ajustará automáticamente al tamaño de su widget hijo más grande, esto significa que, si no especificas un tamaño explícito para la pila, esta se expandirá para acomodar a sus hijos.

Stack es útil cuando se desea superponer elementos en tu interfaz de usuario o cuando necesitamos un diseño personalizado que combine múltiples widgets de manera controlada.

Container²⁴

Se utiliza para crear un contenedor rectangular que puede contener otros widgets, es uno de los widgets más versátiles en Flutter y se utiliza comúnmente para diseñar y personalizar la apariencia de elementos en la interfaz de usuario.

A continuación, se muestra un ejemplo de su codificación:

```
Container(  
  width: 200,  
  height: 100,  
  color: Colors.blue,  
  margin: EdgeInsets.all(10),  
  padding: EdgeInsets.symmetric(horizontal: 20, vertical: 10),
```

²⁴ <https://api.flutter.dev/flutter/widgets/Container-class.html>

```
alignment: Alignment.center,  
child: Text(  
  'Contenedor Personalizado',  
  style: TextStyle(color: Colors.white),  
),  
)
```

Es posible aplicar decoraciones al contenedor utilizando la propiedad *decoration*, esto permite definir el color de fondo, bordes, sombras y otros efectos visuales para personalizar la apariencia del contenedor.

Si se requiere especificar el ancho y el alto del contenedor es posible hacerlo mediante las propiedades *width* y *height*, si no se especifica ningún tamaño, el contenedor se ajustará automáticamente al tamaño de su contenido.

El contenedor admite las propiedades *margin* y *padding*, que permiten controlar el espacio alrededor del contenedor y el espacio entre su contenido y los bordes.

Es posible también, alinear el contenido del contenedor en relación con el propio contenedor utilizando la propiedad *alignment*.

Container admite la aplicación de transformaciones, como rotación, escala y traslación a su contenido. También es posible aplicar opacidades y recortar el contenido del contenedor si es necesario.

Para limitar el tamaño del contenedor se emplea las propiedades *constraints*, lo que puede ser útil cuando se desea que el contenedor tenga un tamaño máximo o mínimo.

También, se puede personalizar la forma del contenedor utilizando la propiedad *clipBehavior* para recortar su contenido de diversas maneras.

GridView²⁵

Es un widget que se utiliza para crear una cuadrícula de elementos en una interfaz de usuario, estos se organizan en filas y columnas, lo que facilita la creación de diseños de cuadrícula flexibles y responsivos en sus aplicaciones.

A continuación, se presenta un ejemplo para su codificación:

```
GridView.count(  
  crossAxisCount: 2,  
  crossAxisSpacing: 10.0,  
  mainAxisSpacing: 10.0,  
  children: List.generate(  
    4,
```

²⁵ <https://api.flutter.dev/flutter/widgets/GridView-class.html>

```
(index) => Container(  
  color: Colors.blue,  
  child: Center(  
    child: Text(  
      'Elemento ${index + 1}',  
      style: const TextStyle(  
        color: Colors.white,  
        fontSize: 18,  
      ),  
    ),  
  ),  
),  
)),  
,
```

Cuyo resultado es el siguiente:

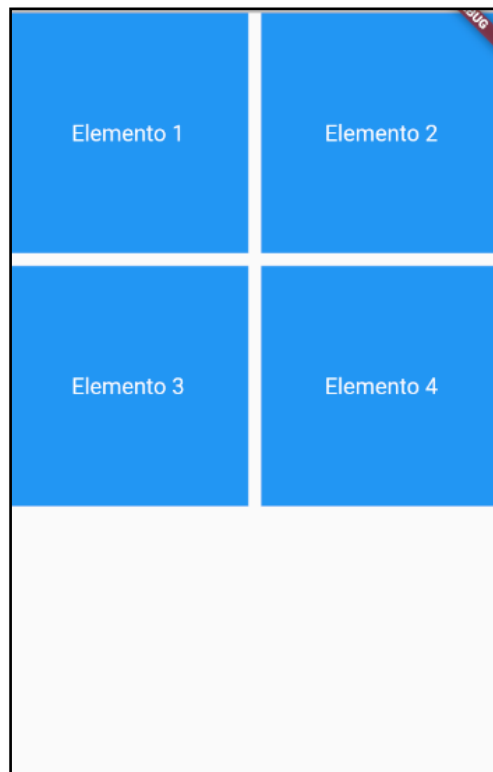


Figura II.8. *GridView con 4 Elementos Organizados en 2 Columnas*

GridView organiza sus elementos en una cuadrícula bidimensional de filas y columnas, permitiendo especificar el número de columnas o ajustar automáticamente el número de columnas según el tamaño de la pantalla. Es factible tener elementos de diferentes tamaños en un *GridView*, facilitando la creación de diseños de cuadrícula donde algunos elementos pueden ocupar más espacio que otros. En caso de que la cuadrícula posea más elementos de los que caben en la pantalla, *GridView* soporta el desplazamiento para acceder a los elementos ocultos.

La personalización de cada elemento de la cuadrícula es posible, al igual que con cualquier otro widget de Flutter, permitiendo la inclusión de elementos diferenciados según las necesidades específicas. *GridView* ofrece control sobre la alineación de los elementos dentro de la cuadrícula mediante propiedades como *crossAxisAlignment* y *mainAxisAlignment*. Además, es posible ajustar el espacio entre las filas y columnas utilizando las propiedades *crossAxisSpacing* y *mainAxisSpacing*.

Como otros widgets de lista en Flutter, *GridView* emplea una técnica de "reciclado de elementos" para optimizar el rendimiento, creando y manteniendo en memoria solo los elementos que son visibles. *GridView* se puede utilizar conjuntamente con elementos infinitos, como los obtenidos de una API, para mostrar grandes cantidades de datos de manera eficiente.

ListView²⁶

Se utiliza para crear una lista de elementos desplazables de manera vertical u horizontal, se puede pensar en *ListView* como una lista de elementos desplazable hacia arriba o hacia abajo (en el caso de una vista vertical) o hacia la izquierda o hacia la derecha (en el caso de una vista horizontal). Suele emplearse este widget para mostrar listas de contenido, como listas de elementos, mensajes, noticias y más. El siguiente es un ejemplo para su codificación:

```
ListView(  
  padding: const EdgeInsets.all(8),  
  children: <Widget>[  
    Container(  
      height: 50, color: Colors.amber[600],  
      child: const Center(child: Text('Item A'))),  
    ),  
    Container(  
      height: 50, color: Colors.amber[500],  
      child: const Center(child: Text('Item B'))),  
    ),  
    Container(  
      height: 50, color: Colors.amber[100],  
      child: const Center(child: Text('Item C'))),  
    ),  
  ],  
)
```

El resultado visual se puede apreciar a continuación:

²⁶ <https://api.flutter.dev/flutter/widgets/ListView-class.html>

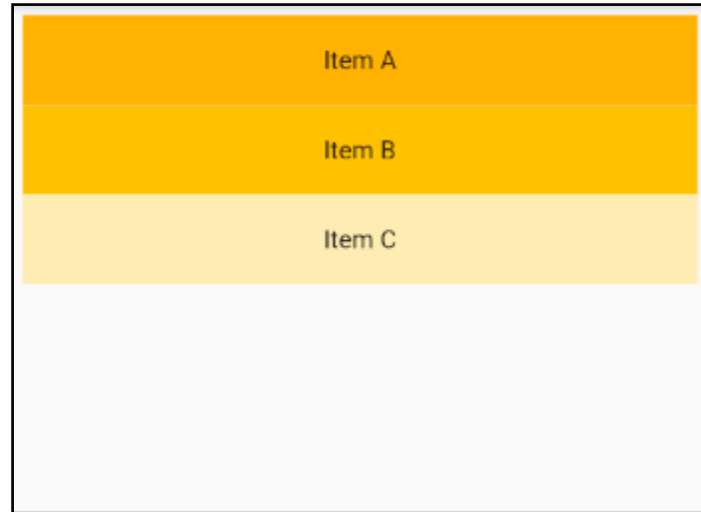


Figura II.9. *ListView* con 3 Elementos

El widget *ListView* permite desplazarse a través de un conjunto de elementos cuando la lista supera la longitud de la pantalla. La personalización de cada elemento de la lista es factible utilizando el constructor *ListView.builder*, el cual facilita la generación dinámica de elementos a partir de una fuente de datos. Para crear una lista estática de elementos, se puede emplear el constructor *ListView*, especificando cada elemento como un widget hijo. *ListView* soporta la alineación de elementos mediante la propiedad *alignment*, permitiendo controlar la disposición de los elementos en el eje principal. La inserción de separadores entre los elementos de la lista se logra con el constructor *ListView.separated*, útil para separar visualmente los elementos de la lista. El control del desplazamiento en un *ListView* se puede realizar mediante un *ScrollController*, lo que posibilita ejecutar acciones personalizadas durante el desplazamiento del usuario. La carga de elementos en la lista conforme el usuario se desplaza hacia abajo (scroll infinito) se puede implementar utilizando técnicas como *ListView.builder* junto con una fuente de datos dinámica. Además, *ListView* admite la creación de vistas personalizadas para elementos de la lista, lo cual permite el diseño de elementos de lista complejos y personalizados.

Flexible²⁷

El widget Flexible es un componente en Flutter que se utiliza para controlar cómo un widget se expande o contrae en un espacio disponible dentro de un *Row* o *Column*. Este widget es parte del sistema de diseño flexible de Flutter y es útil para crear diseños responsivos y dinámicos en la interfaz de usuario. El siguiente código brinda una referencia de cómo utilizarlo:

```
Row(  
  children: [  
    Flexible(  

```

²⁷ <https://api.flutter.dev/flutter/widgets/Flexible-class.html>

```
flex: 2,  
child: Container(  
  color: Colors.blue,  
  height: 100,  
),  
),  
Flexible(  
  flex: 1,  
  child: Container(  
    color: Colors.green,  
    height: 100,  
  ),  
),  
],  
)
```

En este ejemplo, se crea una fila con dos widgets `Flexible`. El primer widget ocupa el doble de espacio que el segundo widget debido a su factor de flexión (*flex: 2* y *flex: 1*, respectivamente). La siguiente imagen ilustra cómo *Flexible* se utiliza para controlar la distribución del espacio en un diseño:



Figura II.10. Resultado de la Aplicación de *Flexible*

La propiedad más importante de `Flexible` es *flex*, que determina la proporción del espacio disponible que el widget debería ocupar en relación con otros widgets flexibles en la misma fila o columna.

Es posible controlar cómo el contenido del widget `Flexible` se alinea dentro del espacio que ocupa utilizando la propiedad *alignment*, esto permite alinear el contenido al principio, al final o en el centro del espacio disponible.

Flexible puede envolver cualquier widget como su hijo, lo que representa la posibilidad personalizar el contenido dentro del widget flexible según las necesidades. Puede ser un *Container*, un *Text*, un *Image*, u otro widget de elección.

Cuando se utiliza *Flexible* junto con otros widgets *Flexible* en un *Row* o *Column*, el espacio disponible se distribuirá entre ellos de acuerdo con sus factores de flexión, esto permite que los widgets se expandan o contraigan dinámicamente en función del espacio disponible.

Expanded²⁸

El widget *Expanded* es un componente que se utiliza para indicar que un widget hijo debe ocupar todo el espacio disponible dentro de un *Row* o *Column*. *Expanded* es una forma abreviada de utilizar el widget *Flexible* con un factor de flexión (*flex*) de 1.

El siguiente es un ejemplo simple de cómo se utiliza *Expanded* en una *Column*:

```
Column(  
  children: [  
    const Text('Elemento 1'),  
    const Text('Elemento 2'),  
    Expanded(  
      child: Container(  
        color: Colors.blue,  
        height: 100,  
        child: const Center(  
          child: Text(  
            'Contenido Expandido',  
            style: TextStyle(  
              color: Colors.white,  
              fontSize: 18,  
            ),  
          ),  
        ),  
      ),  
    ),  
    const Text('Elemento 3'),  
  ],  
)
```

En este ejemplo, se ha creado una *Column* con varios elementos, el tercer elemento es un *Container* envuelto en un widget *Expanded*, debido a la presencia de *Expanded*, el *Container* ocupará todo el espacio vertical disponible en la columna, lo que significa que se expandirá para llenar el espacio entre "Elemento 2" y "Elemento 3", lo que es especialmente útil cuando se desea que un widget, como un *Container*, ocupe todo el espacio restante en un diseño flexible.

²⁸ <https://api.flutter.dev/flutter/widgets/Expanded-class.html>

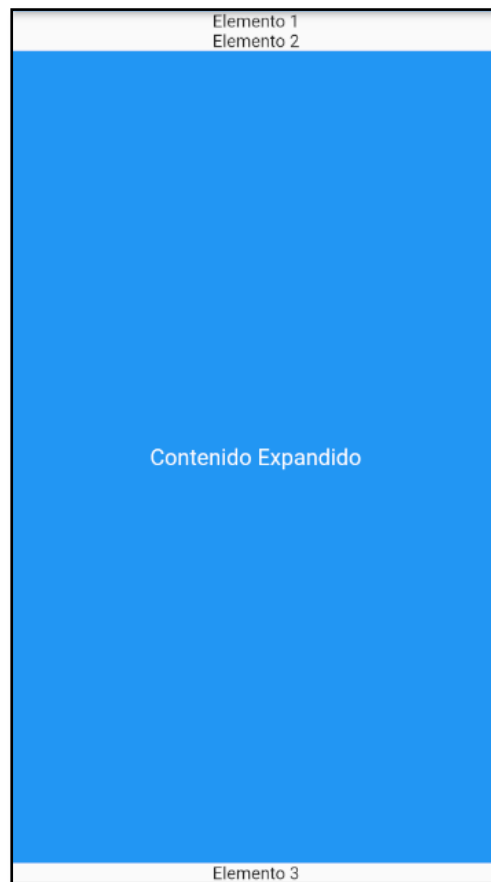


Figura II.11. Resultado de la Aplicación de Container

Cuando se coloca un widget *Expanded* en un *Row* o *Column*, el espacio disponible restante en esa fila o columna se asigna al widget *Expanded*, esto hace que el widget *Expanded* ocupe todo el espacio disponible, incluso si hay otros widgets en la misma fila o columna.

Es posible utilizar múltiples widgets *Expanded* en una fila o columna junto con otros widgets no flexibles para crear diseños complejos y responsivos.

Expanded puede envolver cualquier widget como su hijo, lo que te permite personalizar el contenido dentro del espacio que ocupa.

Al igual que *Flexible*, *Expanded* también admite la propiedad *alignment* para controlar cómo se alinea el contenido dentro del espacio ocupado.

LayoutBuilder²⁹

Es un widget de Flutter que proporciona información sobre las restricciones de diseño del espacio disponible para su widget hijo. Este widget es especialmente útil cuando necesitas personalizar el diseño o el tamaño de un widget en función del espacio disponible en el diseño de la interfaz de usuario.

²⁹ <https://api.flutter.dev/flutter/widgets/LayoutBuilder-class.html>

Cuando se envuelve un widget con *LayoutBuilder*, el widget hijo tiene acceso a las restricciones de diseño del espacio disponible, estas restricciones incluyen el ancho y la altura máximos permitidos para el widget hijo.

Se puede utilizar la información de las restricciones proporcionadas por *LayoutBuilder* para personalizar el tamaño, la posición u otros aspectos del widget hijo de acuerdo con el espacio disponible.

LayoutBuilder es especialmente útil para crear diseños responsivos en Flutter, donde el contenido se adapta automáticamente al tamaño de la pantalla o al espacio disponible en la interfaz de usuario.

Es posible anidar múltiples widgets *LayoutBuilder* para construir diseños complejos y anidados que respondan a diferentes restricciones de diseño en diferentes niveles.

El siguiente es un ejemplo simple de cómo se utiliza *LayoutBuilder* en Flutter:

```
LayoutBuilder(  
  builder: (BuildContext context, BoxConstraints constraints) {  
    return Container(  
      color: Colors.blue,  
      width: constraints.maxWidth,  
      height: constraints.maxHeight,  
      child: Center(  
        child: Text(  
          'Contenido Personalizado',  
          style: TextStyle(  
            color: Colors.white,  
            fontSize: 18,  
          ),  
        ),  
      ),  
    );  
  },  
)
```

MediaQuery³⁰

Es una clase de Flutter que proporciona información sobre el contexto de la pantalla o el dispositivo en el que se está ejecutando la aplicación. Se utiliza para obtener datos relacionados con el tamaño de la pantalla, la orientación, la densidad de píxeles (píxeles por pulgada) y otros aspectos del entorno de ejecución de la aplicación. Es especialmente útil para crear aplicaciones con diseños responsivos, donde el contenido y la interfaz de usuario se adaptan automáticamente al tamaño y la orientación de la pantalla del dispositivo.

La clase *MediaQuery* se utiliza para ajustar el tamaño de fuente y otros aspectos de la apariencia de la aplicación en función del tamaño de la pantalla o de las preferencias del usuario.

³⁰ <https://api.flutter.dev/flutter/widgets/MediaQuery-class.html>

Se emplea para detectar el tipo de dispositivo en el que se está ejecutando la aplicación, como un teléfono móvil, una tableta o un dispositivo de escritorio.

MediaQuery permite conocer la orientación actual del dispositivo, lo que es útil para realizar cambios en la interfaz de usuario en respuesta a cambios en la orientación.

Se puede utilizar *MediaQuery* para establecer puntos de diseño específicos en la aplicación que se activan cuando el ancho de la pantalla alcanza ciertos valores.

Con *MediaQuery* se tiene acceso al tema actual de la aplicación, lo que le permite personalizar la apariencia en función de las preferencias del usuario.

Para utilizar *MediaQuery* en una aplicación, se puede obtener una referencia a ella utilizando *MediaQuery.of(context)*, donde *context* es el contexto actual de la aplicación. A partir de esa referencia, es posible acceder a diferentes propiedades y métodos para obtener información sobre el entorno de ejecución de la aplicación y ajustar la interfaz de usuario en consecuencia.

Por ejemplo, se puede obtener el tamaño de la pantalla actual de esta manera:

```
MediaQueryData mediaQueryData = MediaQuery.of(context);  
Size screenSize = mediaQueryData.size;
```

En resumen, los layouts en Flutter son fundamentales para el diseño de cualquier aplicación, ya que definen la estructura espacial y visual de la interfaz de usuario. Una comprensión sólida de los widgets de layout y cómo interactúan entre sí es esencial para el desarrollo efectivo de aplicaciones en Flutter.

2.2.7 Dialogs

Los diálogos son una forma de presentar información u opciones al usuario dentro de una ventana emergente, son especialmente útiles para captar la atención del usuario, pedir confirmaciones o mostrar información crítica, se manejan principalmente a través de dos widgets: *SimpleDialog* y *AlertDialog*.

SimpleDialog³¹

Se utiliza para mostrar una lista de opciones entre las que el usuario puede elegir, puede incluir un título opcional y una lista de hijos (*children*), cada hijo suele ser un *SimpleDialogOption*.

A continuación, se explica detalladamente cómo emplear un *SimpleDialog*:

Para crear un *SimpleDialog* generalmente se utiliza el método *showDialog*, que es una función asíncrona que devuelve un Future que se resuelve cuando el diálogo se cierra, sobre esto último, se explicará con detalle en el Capítulo 5, por ahora, simplemente se tratará como una función normal.

³¹ <https://api.flutter.dev/flutter/material/SimpleDialog-class.html>

```
showDialog(  
  context: context,  
  builder: (BuildContext context) {  
    return SimpleDialog(  
      title: Text('Título del Diálogo'),  
      children: <Widget>[  
        // Aquí van las opciones del diálogo  
      ],  
    );  
  },  
);
```

El parámetro *title* permite establecer un título para el diálogo. Es opcional, pero es una buena práctica incluirlo para dotarle de claridad a la intención del diálogo.

```
title: Text('Elige una opción'),
```

Dentro de *SimpleDialog*, se puede usar el parámetro *children* para añadir una lista de opciones, generalmente representadas por *SimpleDialogOption*.

```
children: <Widget>[  
  SimpleDialogOption(  
    onPressed: () { Navigator.pop(context, 'Opción 1'); },  
    child: const Text('Opción 1'),  
  ),  
  SimpleDialogOption(  
    onPressed: () { Navigator.pop(context, 'Opción 2'); },  
    child: const Text('Opción 2'),  
  ),  
  // Puede añadir más opciones aquí  
,  
],
```

Cada *SimpleDialogOption* puede tener un *onPressed* para manejar la acción cuando se selecciona esa opción. Al seleccionar una opción, generalmente se cierra el diálogo llamando a *Navigator.pop(context)*, también, es posible devolver un valor que indique la selección del usuario.

El aspecto del *SimpleDialog* y sus opciones (por ejemplo, el color de fondo, el estilo de texto) pueden personalizarse utilizando los parámetros disponibles o envolviéndolos en widgets adicionales. El tamaño del diálogo se adapta al contenido, pero también se puede controlar envolviendo el *SimpleDialog* en un *Container* o utilizando *ConstrainedBox*. Es importante asegurar que las opciones sean claras y accesibles para todos los usuarios, incluyendo aquellos que utilizan lectores de pantalla.

AlertDialog³²

El *AlertDialog* es un widget utilizado para mostrar información importante que requiere la confirmación o atención del usuario. A diferencia del *SimpleDialog*, que se centra en ofrecer una lista de opciones, el *AlertDialog* se considera más adecuado para situaciones críticas como confirmaciones, advertencias o para comunicar mensajes de error. Este widget incluye un área de acciones para botones que, por lo general, representan acciones como 'Aceptar' o 'Cancelar'.

Diseñado para mostrar un mensaje específico (content), el *AlertDialog* es menos flexible en términos de variedad de contenido y presenta un estilo más enfocado en la alerta o advertencia. El diseño está estructurado de manera que enfatiza el contenido del mensaje y las acciones, diferenciándose así del widget *SimpleDialog* en su enfoque y estructura.

```
AlertDialog(  
  title: Text('Título'),  
  content: Text('Mensaje de alerta'),  
  actions: <Widget>[  
    TextButton(  
      onPressed: () { /* Acción */ },  
      child: Text('Cancelar'),  
    ),  
    // Más botones...  
  ],  
)
```

En resumen, mientras *SimpleDialog* es más adecuado para presentar una serie de opciones en un formato más flexible, *AlertDialog* es ideal para situaciones que requieren una atención inmediata o una acción de confirmación, con un enfoque más directo en el mensaje y las acciones del usuario. La elección entre uno y otro depende del contexto y el objetivo de la interacción con el usuario dentro de tu aplicación Flutter.

2.3 Navegación

Las aplicaciones móviles suelen tener múltiples pantallas. En Flutter, un elemento clave para la navegación entre pantallas es el widget *Navigator*, el cual se encarga de gestionar los cambios de pantalla y mantener un historial de las pantallas visitadas para permitir al usuario retroceder entre ellas, si la aplicación lo permite.

2.3.1 El Widget Navigator

Según Napoli, p. (2020, p. 178) “El widget Navigator gestiona una pila de rutas para moverse entre páginas. Opcionalmente, puedes pasar datos a la página de destino y de regreso a la página original.” [Traducción propia].

³² <https://api.flutter.dev/flutter/material/AlertDialog-class.html>

De acuerdo a lo anterior, es posible mencionar que el widget `Navigator` maneja la navegación entre páginas en una aplicación, permitiendo la transición entre diferentes pantallas a través de una pila de rutas. Facilita el envío de datos entre páginas y soporta gestos de navegación nativos para iOS y Android. Trabaja con métodos como `push`, `pushNamed` y `pop` para ofrecer una gestión flexible y eficiente del flujo de navegación.

Para comenzar a navegar entre páginas, se debe utilizar el método `Navigator.push`. El siguiente código ofrece una referencia de como emplearlo:

```
Navigator.push(  
  context,  
  MaterialPageRoute(  
    fullscreenDialog: true,  
    builder: (context) => const SecondScreen(),  
  ),  
);
```

El ejemplo anterior muestra cómo utilizar el método `Navigator.push` para navegar a la página `SecondScreen`. El método `push` pasa los argumentos `BuildContext` y `Route`.

Para empujar una nueva ruta, se crea una instancia de la clase `MaterialPageRoute` que reemplaza la pantalla con la transición de animación dependiendo de la plataforma (iOS o Android). En el ejemplo, la propiedad `fullscreenDialog` se establece en `true` para presentar `SecondScreen` como un cuadro de diálogo modal de pantalla completa. Al establecer la propiedad `fullscreenDialog` en `true`, la barra de aplicaciones de la página `SecondScreen` incluye automáticamente un botón de cierre.

```
Navigator.pop(context);
```

Para cerrar la página y navegar de regreso es posible emplear el método `Navigator.pop`. Se invoca al método `Navigator.pop(context)` pasando el argumento `BuildContext` y la página se cierra deslizándose desde la parte superior de la pantalla hacia la parte inferior.

Si lo que se desea es pasar un valor de vuelta a la página anterior, se debe emplear el método de la siguiente manera:

```
Navigator.pop(context, 'Done');
```

El método `Navigator.pop(context, 'Done')` permite cerrar la pantalla actual (o ruta) y, opcionalmente, pasar datos de vuelta a la pantalla anterior. En este caso, el texto `'Done'` es el dato que se está pasando de vuelta.

Ejercicio: Navegación entre Páginas.

A continuación, se desarrollará un ejemplo claro y funcional de navegación básica en Flutter utilizando el widget *Navigator*, esto con la finalidad de repasar lo estudiado anteriormente y tener una perspectiva clara e integral de lo mencionado.

Parte 1: Punto de entrada y configuración inicial

```
import 'package:flutter/material.dart';  
void main() {  
  runApp(const MaterialApp(  
    title: 'Navegación con Navigator',  
    home: MainScreen(),  
  ));  
}
```

El código anterior, al margen de lo que ya se explicó en relación a *import*, el *main* y *runApp*; establece el título de la aplicación y se define la pantalla inicial (*home*) con *MainScreen*.

Parte 2: Definición de MainScreen

```
class MainScreen extends StatelessWidget {  
  const MainScreen({super.key});  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(title: const Text('Pantalla Principal')),  
      body: Center(  
        child: ElevatedButton(  
          child: const Text('Ir a la Segunda Pantalla'),  
          onPressed: () {  
            Navigator.push(  
              context,  
              MaterialPageRoute(  
                fullscreenDialog: true,  
                builder: (context) => const SecondScreen(),  
              ),  
            );  
          },  
        ),  
      ),  
    );  
  }  
}
```

La clase *MainScreen* es un *StatelessWidget* que representa la pantalla principal de la aplicación. La vista se construye a partir de un widget *Scaffold* que proporciona la estructura básica de la interfaz de usuario con un *AppBar* y un área de contenido (*body*).

Se ubica un botón para navegar al centro de la pantalla con el widget *Center*, el cual es un widget *ElevatedButton*. Al presionarlo, se inicia la navegación a *SecondScreen*.

El botón utiliza el *Navigator* para colocar *SecondScreen* en el tope de la pila de navegación. Se envuelve en un *MaterialPageRoute*, y se utiliza *fullscreenDialog: true* para que la pantalla se deslice desde abajo como un diálogo en pantalla completa.

Parte 3: Definición de *SecondScreen*

```
class SecondScreen extends StatelessWidget {  
  const SecondScreen({super.key});  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(title: const Text('Segunda Pantalla')),  
      body: Center(  
        child: ElevatedButton(  
          child: const Text('Regresar a la Pantalla Principal'),  
          onPressed: () {  
            Navigator.pop(context);  
          },  
        ),  
      ),  
    );  
  }  
}
```

La clase *SecondScreen* es otro *StatelessWidget* que actúa como la segunda pantalla en la navegación.

La estructura de la pantalla es similar a *MainScreen*, utiliza *Scaffold* para su estructura, incluyendo una *AppBar* y un botón centrado en el *body*.

Al presionar el botón, se activa *Navigator.pop*, lo que cierra *SecondScreen* y regresa a *MainScreen*. Aquí, *Navigator.pop* se utiliza para "retirar" la pantalla actual del tope de la pila de navegación, volviendo a la pantalla anterior.

Aunque la sintaxis pueda parecer un poco intimidante al principio, en realidad todo tiene sentido. Ten en cuenta que lo realizado claramente aplica un enfoque imperativo para la navegación; ya que se está instruyendo al navegador a cambiar de página mediante el uso de *push* y *pop*.

Existe otra forma de utilizar el enfoque imperativo, que implica el uso de rutas con nombre, la cual se verá a continuación.

2.3.2 Rutas con nombre

Las rutas con nombre constituyen una parte importante de la navegación, permitiendo la identificación de la ruta por su administrador, el widget *Navigator*. Se puede definir una serie de rutas con nombres asociados a cada una de ellas, esto proporciona un nivel de abstracción al significado de una ruta y una pantalla. Además, pueden ser utilizadas en una estructura de ruta; en otras palabras, pueden ser consideradas como subrutas de la misma manera en que se estructura una URL web.

El ejercicio anterior de navegación fue sencillo, pero puede dificultarse en una aplicación que requiera una mayor cantidad de vistas, por esa razón, se tiene una alternativa que permite organizar mejor la estructura de navegación, para ello se hace uso de rutas con nombre, lo que permite las siguientes ventajas:

- Organizar las pantallas de una manera clara
- Centralizar la creación de las pantallas
- Pasar parámetros a las pantallas

Las rutas con nombre se especifican en el widget *MaterialApp*, para ello, en primer lugar, se debe incluir en el widget *MaterialApp* una propiedad de rutas (*routes*), que puede tener un aspecto similar al siguiente:

```
routes: {  
  '/': (context) => const MainScreen(),  
  '/screen2': (context) => const SecondScreen(),  
},
```

En el código presentado, se ha especificado que existen dos rutas disponibles dentro de la aplicación: la ruta “/”, que utiliza el widget *MainScreen* para dibujar la “Pantalla Principal”, y la ruta *‘/screen2’*, que emplea el widget *SecondScreen* para su visualización. Al intentar ejecutar el código en su estado actual, se producirá un error debido a que se han configurado tanto la ruta “/” como la propiedad *home* en *MaterialApp*. La ruta “/” es considerada una ruta especial equivalente al inicio de la aplicación, lo que resulta en la definición de la propiedad *home* en dos ocasiones, generando confusión en Flutter sobre cuál debe utilizarse. Por lo tanto, al usar la ruta “/”, se debe eliminar la propiedad *home* y probar la aplicación, la cual debería funcionar correctamente en este punto.

Para finalizar, se sugiere modificar la navegación para que utilice la ruta especificada. En el *ElevatedButton* de *MainScreen*, se debe editar el evento *onPressed* de manera que se ajuste a la nueva configuración de navegación propuesta.

```
onPressed: () {  
  Navigator.pushNamed(context, '/screen2');  
},
```

Note cuánto más limpio se siente el código y cuán clara es la intención de la navegación, la creación de un *MaterialPageRoute* ahora es implícita. Lo que resta es ahora es verificar que el flujo de navegación siga funcionando correctamente.

Por otro lado, *pop()* continúa funcionando como antes, ya que *pushNamed* solo agrega una pantalla a la pila, por lo que el *pop* simplemente eliminará esa pantalla como se esperaba.

Paso de argumentos.

Seguramente se notó que el título para *SecondScreen* siempre será el mismo, esto a menudo no es el comportamiento que se desearía; seguramente se esperaría que la pantalla de llamada pueda establecer los argumentos de la pantalla a la que se está navegando.

Para resolver esta necesidad, el método *pushNamed* también acepta argumentos que se pasan a la ruta, en ese caso el código se verá de la siguiente manera:

```
onPressed: () {  
  Navigator.of(context)  
    .pushNamed('/screen2', arguments: "Argumento Pantalla Secundaria");  
},
```

Sin embargo, la vida no es tan simple ahora y ya no se puede usar el parámetro *routes* de *MaterialApp*. En su lugar, se deberá usar el parámetro *onGenerateRoute* para pasar la configuración al widget *SecondScreen*.

En el widget *MaterialApp*, elimina el parámetro *routes* y agrega *onGenerateRoute* para que se vea de la siguiente manera:

```
onGenerateRoute: (settings) {  
  if (settings.name == '/') {  
    return MaterialPageRoute(  
      builder: (context) =>  
        const MainScreen()  
    );  
  } else if (settings.name == '/screen2') {  
    return MaterialPageRoute(  
      builder: (context) =>  
        SecondScreen(title: settings.arguments as String)  
    );  
  }  
  return MaterialPageRoute(  
    builder: (context) =>  
      const MainScreen() // Una pantalla genérica de "No encontrado"  
    );  
},
```

El código de *onGenerateRoute* examina *settings.name* para ver qué ruta se seleccionó y luego devuelve un *MaterialPageRoute* con los detalles requeridos para esa ruta, esto seguramente resulta familiar al código escrito cuando se empleo el método *push* en lugar de *pushNamed*, sin embargo, presenta la desventaja obvia de que se perdió la seguridad de tipos en *settings.arguments* y que se deberá confiar en que la conversión a *String* funcione correctamente.

Quizá con el anterior inconveniente surge la pregunta ¿Debería usar rutas con nombre?, la elección de utilizar rutas con nombre es personal y no hay una respuesta correcta.

Personalmente, prefiero utilizar el método *push* porque tiene una fuerte seguridad de tipos en los parámetros del constructor del widget dentro de la ruta. Sin embargo, si no se está pasando argumentos a las rutas o se prefiere tener una gestión centralizada de las rutas, entonces puede optar por usar rutas con nombre.

Ya en este punto, se revisó como desplazarse a una ruta utilizando *push* y *pushNamed* y cómo pasar argumentos a esas rutas. Sin embargo, habrá situaciones en las que se requiera devolver un resultado a la pantalla de llamada durante un *pop()*, por ello se presenta eso a continuación.

2.3.3 Obtener resultados de una ruta

Al empujar una ruta en la navegación, es posible que se desee esperar algo a cambio de ella; por ejemplo, al solicitar información al usuario en una nueva ruta, se puede capturar el valor devuelto a través del parámetro de resultado del método *pop()*.

El método *push* y sus variantes devuelven un *Future*. Dicho *Future* se resuelve cuando la ruta se elimina de la pila de navegación, siendo el valor del *Future* el parámetro de resultado del método *pop()*. Se observa que,

además de pasar argumentos a una nueva ruta, es posible realizar el proceso inverso; es decir, en lugar de enviar un mensaje a la segunda pantalla, se puede recibir un mensaje cuando esta regresa.

Para actualizar *SecondScreen* y permitir que el usuario tome una decisión, en el método *build*, se actualiza el widget *Center* para que el hijo sea una *Column* que contenga dos *ElevatedButtons* como se puede apreciar a continuación:

```
class SecondScreen extends StatelessWidget {
  final String title;
  const SecondScreen({super.key, required this.title});
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text(title)),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            ElevatedButton(
              child: const Text("True"),
              onPressed: () {
                Navigator.of(context).pop(true);
              },
            ),
            ElevatedButton(
              child: const Text("False"),
              onPressed: () {
                Navigator.of(context).pop(false);
              },
            ),
          ],
        ),
      ),
    );
  }
}
```

Se observará que, aparte del ligero cambio en el árbol de widgets para acomodar el *ElevatedButton* adicional, ahora *pop()* toma un argumento. En este caso, es un booleano, pero cualquier tipo puede ser devuelto a través del método *pop*. Es necesario actualizar el código en el evento *onPressed* del *ElevatedButton* en *MainScreen* para que pueda recibir el valor devuelto.

A continuación, se muestra cómo se podría realizar esta actualización con un código de ejemplo. Cabe destacar que el ejemplo ha vuelto a utilizar *push* en lugar de *pushNamed* para la navegación. Se invita a adaptar libremente el código anterior que utiliza rutas con *push*:

```
onPressed: () async {
  final result = await Navigator.push(
    context,
    MaterialPageRoute(
      fullscreenDialog: true,
      builder: (context) => const SecondScreen(title: "Segunda Pantalla"),
    ),
  );
  if (result != null) {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text("$result")),
    );
  }
},
```

El resultado de *push* es un *Future*, lo que implica que el código debe esperar el resultado, especificándolo mediante la palabra clave *await*. Esto conlleva la necesidad de actualizar el método de la función anónima proporcionada a *onPressed* para indicar que es una función asincrónica, lo cual se realiza con la palabra clave *async* entre la lista de parámetros, que está vacía y el cuerpo del método entre llaves.

Posteriormente, se emplea una característica de Flutter, *SnackBar*, para exhibir el resultado en pantalla mediante *ScaffoldMessenger*, que es un *InheritedWidget*. Para ello, se utiliza la sintaxis *<tipo>.of()* para localizarlo y posteriormente mostrar un *SnackBar* (una notificación que se desliza hacia arriba desde la parte inferior de la pantalla). El método *showSnackBar* simplemente requiere un widget *SnackBar* como parámetro, en el cual se especifica tener un widget hijo del tipo *Text* que contiene el valor del resultado.

Hasta este punto, se ha abordado el uso de “Navigator 1.0”, un sistema de navegación completamente funcional e intuitivo que facilita la navegación de los usuarios. El empleo del widget *Navigator*, junto con las rutas, se presenta como fácil de comprender e implementar. Aunque “Navigator 1.0” ha sido suficiente para numerosas aplicaciones y raramente presenta limitaciones, existen situaciones, especialmente en aplicaciones web, donde podría no ajustarse completamente a los requerimientos, recomendándose en esos casos revisar “Navigator 2.0” para trabajos con Flutter en la web.

2.3.4 Animar transiciones

Para que el usuario disfrute la experiencia dentro de una aplicación, es necesario evitar las transiciones bruscas de pantalla, ya que pueden afectar la fluidez.

Como se ha observado, *MaterialPageRoute* y *CupertinoPageRoute* son clases que agregan una ruta al navegador. Es posible que se haya notado, mientras se experimentaba con la aplicación de ejemplo, que añaden una transición entre la ruta antigua y la nueva. Estas transiciones se ajustan a las configuraciones predeterminadas de la plataforma, pero también pueden personalizarse. Para ello, se puede utilizar *PageRouteBuilder*.

PageRouteBuilder es una clase que permite crear rutas de página con transiciones animadas personalizadas. Es particularmente útil cuando se necesita más control sobre cómo se presentará una nueva pantalla en su aplicación, en comparación con las transiciones predeterminadas proporcionadas por *MaterialPageRoute* o *CupertinoPageRoute*.

¿Qué es lo que se puede hacer con *PageRouteBuilder*? Es posible crear transiciones únicas que no se encuentran en las rutas estándar de Flutter (*MaterialPageRoute*, *CupertinoPageRoute*), como:

- **Deslizamientos (Slides):** Mover la página entrante desde cualquier dirección (izquierda, derecha, arriba, abajo).
- **Fundidos (Fades):** Hacer que la página entrante aparezca gradualmente desde la transparencia.
- **Escala:** Hacer que la página entrante se agrande o se reduzca desde un punto central o desde un extremo.
- **Rotaciones:** Rotar la página entrante en el eje X, Y o Z.
- **Combinaciones:** Combinar diferentes animaciones, como deslizar y desvanecer simultáneamente.

En resumen, *PageRouteBuilder* ofrece un gran poder para personalizar la navegación en una aplicación, permitiendo crear experiencias únicas y atractivas para los usuarios. Para lograr lo mencionado es importante conocer sus componentes, los cuales son:

- *pageBuilder*: Es una función requerida que construye el contenido de la nueva página, recibe tres parámetros: el *BuildContext* y dos animaciones (*Animation<double>*), una para la animación de la nueva pantalla (*animation*) y otra para la animación de la pantalla anterior (*secondaryAnimation*).
- *transitionsBuilder*: Es otra función requerida que define la animación de transición, también recibe cuatro parámetros: el *BuildContext*, las dos animaciones mencionadas anteriormente y el *child*, que es el widget retornado por *pageBuilder*. Dentro de esta función, se puede definir cómo se animará el *child* durante la transición.
- *transitionDuration*: Es opcional y define la duración de la transición, si no se especifica, Flutter usa un valor predeterminado.
- *reverseTransitionDuration*: Similar a *transitionDuration*, pero para la duración de la animación cuando la página se cierra.

- *opaque*: Un booleano que indica si la ruta es opaca o no, una ruta opaca obstaculiza ver las rutas debajo de ella.
- *barrierColor*: El color del modal *barrier* que se muestra cuando la ruta está en el top de la pila.
- *barrierDismissible*: Un booleano que indica si puedes descartar la ruta tocando el *barrier*.
- *barrierLabel*: Etiqueta semántica para el *barrier* (importante para accesibilidad).
- *maintainState*: Un booleano que indica si esta ruta conserva su estado cuando no es visible.

PageRouteBuilder es una herramienta poderosa para personalizar transiciones, pero su uso indebido puede llevar a interfaces de usuario confusas o a una mala experiencia de usuario. Se debe tener el cuidado de probar las transiciones en diferentes dispositivos y orientaciones para garantizar una experiencia fluida y coherente.

A continuación, se muestra un ejemplo de cómo utilizar *PageRouteBuilder* para crear una transición de deslizamiento (slide) al navegar hacia una nueva página:

En este ejemplo, se explora cómo implementar transiciones personalizadas utilizando *PageRouteBuilder*. Mediante un ejemplo práctico, se mostrará cómo implementar una animación de deslizamiento suave desde la derecha al navegar hacia una nueva página.

El ejemplo consta de dos pantallas principales:

- *MainPage*: La pantalla inicial de la aplicación.
- *NewPage*: La pantalla de destino a la que se navegará.

Se inicia con la clase *NewPage* por su simplicidad, dado que contiene elementos básicos sin presentar ninguna novedad.

```
class NewPage extends StatelessWidget {
  const NewPage({super.key});
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('New Page'),
      ),
      body: const Center(
        child: Text('Welcome to the New Page!', style: TextStyle(fontSize: 24)),
      ),
    );
  }
}
```

Se continúa con la *MainPage*, que se constituye en la parte central de este ejemplo, enfocándose específicamente en el método *navigateToNewPage* que se invoca desde el botón ubicado en la parte central

de esta pantalla. Al presionar este botón, se activará la transición a *NewPage* utilizando una animación de deslizamiento personalizada.

```
class MainPage extends StatelessWidget {
  const MainPage({super.key});
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Main Page'),
      ),
      body: Center(
        child: ElevatedButton(
          onPressed: () => navigateToNewPage(context),
          child: const Text('Go to New Page'),
        ),
      ),
    );
  }
  // Función para navegar a NewPage con una animación personalizada
  void navigateToNewPage(BuildContext context) {
    Navigator.of(context).push(PageRouteBuilder(
      pageBuilder: (context, animation, secondaryAnimation) => const NewPage(),
      transitionsBuilder: (context, animation, secondaryAnimation, child) {
        const begin = Offset(1.0, 0.0); // Inicia desde la derecha
        const end = Offset.zero;
        const curve = Curves.ease;
        var tween =
          Tween(begin: begin, end: end).chain(CurveTween(curve: curve));
        var offsetAnimation = animation.drive(tween);
        return SlideTransition(
          position: offsetAnimation,
          child: child,
        );
      },
    ));
  }
}
```

Es necesario analizar con detalle la función *navigateToNewPage*, la cual personaliza la transición de página. Se desglosará este método detalladamente para entender cada componente y su función.

Inicialmente, la función recibe *BuildContext* como argumento, que es una referencia al contexto del widget desde el cual se llama la función. Este contexto es imprescindible para interactuar con el *Navigator*.

El método `Navigator.of(context).push` inicia la navegación hacia una nueva ruta, `Navigator` es una clase esencial en Flutter que gestiona la pila de rutas (pantallas) en una aplicación, añadiendo una nueva ruta a la pila.

El constructor `PageRouteBuilder` permite definir una ruta de página con transiciones de animación personalizadas. Aunque ya se han descrito sus componentes anteriormente, posee dos propiedades principales: `pageBuilder` y `transitionsBuilder`, las cuales se emplean en este ejemplo.

El componente `pageBuilder` define la página a la que se navega. Y maneja los siguientes argumentos:

- `context`: El contexto del builder.
- `animation`: La animación para la transición de la página entrante.
- `secondaryAnimation`: La animación para la transición de la página saliente.

El componente `transitionsBuilder` define cómo se anima la transición entre la página actual y `NewPage`. Los argumentos que maneja son los mismos que `pageBuilder`, más el `child`, que es el widget retornado por `pageBuilder`. En este componente es donde se define el proceso de animación que en este caso comprende:

- **Offset de Inicio y Fin:**

```
const begin = Offset(1.0, 0.0);
```

La animación comienza desde el lado derecho de la pantalla. `Offset(1.0, 0.0)` significa que el punto de inicio está 100% a la derecha del borde de la pantalla.

```
const end = Offset.zero;
```

La animación termina en el centro de la pantalla. `Offset.zero` es equivalente a `Offset(0.0, 0.0)`.

- **Curva de Animación:**

```
const curve = Curves.ease;
```

Define cómo varía la velocidad de la animación, `Curves.ease` es una curva común que comienza lento, acelera en el medio y termina lentamente.

- **Interpolación:**

```
var tween = Tween(begin: begin, end: end).chain(CurveTween(curve: curve));
```

Inicialmente interpola entre los *Offset* de inicio y fin, posteriormente con el método *chain(CurveTween(curve: curve))* se aplica la curva de animación con el metodo *CurveTween* y la variable *curve* previamente asignada.

- **Animación de Desplazamiento:**

```
var offsetAnimation = animation.drive(tween);
```

Conduce la animación basada en el *Tween* configurado, esto vincula la animación de desplazamiento con el ciclo de vida de la transición de la página.

- **Transición de deslizamiento:**

```
return SlideTransition(  
  position: offsetAnimation,  
  child: child,  
);
```

Finalmente, se aplica la animación de deslizamiento. En la cual *position* usa *offsetAnimation* como la fuente de la animación de deslizamiento. Y *child* es el widget que se animará, es decir, la nueva página (*NewPage*).

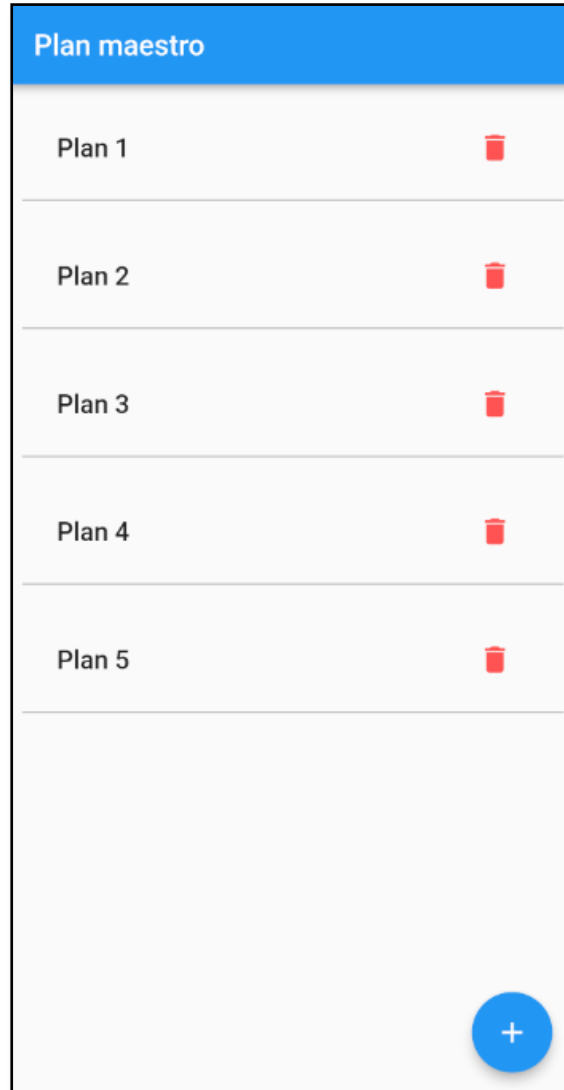
Esta función es un claro ejemplo de cómo Flutter ofrece un control detallado y flexible sobre las animaciones y transiciones en las aplicaciones móviles. Al personalizar transiciones como en *navigateToNewPage*, se puede mejorar significativamente la experiencia del usuario, haciendo que la navegación entre pantallas sea más fluida e intuitiva.

2.4 Ejercicio: UI para una aplicación de lista de planes

Este ejercicio proporciona una base sólida para el diseño de interfaces de usuario (UI), con el objetivo principal de aplicar los conceptos estudiados hasta este punto. Se sentarán las bases para una aplicación práctica que se implementará progresivamente a lo largo de este libro. En esta sección, se iniciará con el desarrollo de la interfaz de usuario (UI) de la lista de planes.

2.4.1 Instrucciones

El ejercicio de esta sección está diseñado para ayudar a comprender cómo diseñar y organizar los elementos de la UI, además de cómo aplicar estilos para crear una experiencia de usuario atractiva. El resultado esperado al finalizar el ejercicio se presenta a continuación.

*Figura II.12. UI Plan Maestro***Requisitos Previos.**

Para empezar con el ejercicio se debe contar con lo siguiente:

- Tener en funcionamiento el Flutter SDK y un editor de código instalados (contenido abordado en la Sección 3 del Capítulo I).
- Un emulador o dispositivo físico para probar la aplicación (contenido explicado en la Sección 3 del Capítulo I).
- Saber cómo crear y ejecutar una aplicación (explicado en la Sección 4 del Capítulo 1).
- Conocimientos básicos de Dart (estudiado en la Sección 2 del Capítulo 1).
- Entendimiento de Stateless Widgets (revise la Sección 1 del presente Capítulo).
- Conocimientos de widgets básicos (abarcado en la Sección 2 del presente Capítulo).
- Entendimiento de la navegación (incluido en la Sección 3 del presente capítulo).

2.4.2 Construcción de la aplicación

La siguiente es la estructura de carpetas y archivos que se debe tener en la aplicación.

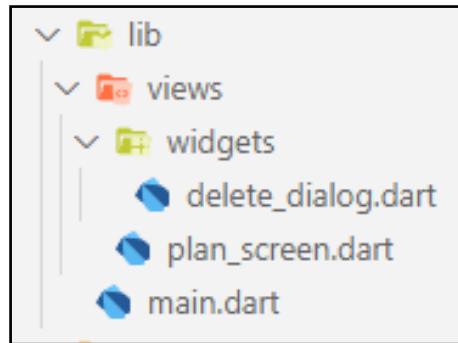


Figura II.13. Estructura del directorio lib de la aplicación

Es necesario aclarar que “lib” tiene como hijos a “views” y “main.dart”. La carpeta “views” se utiliza frecuentemente para alojar todos los recursos desarrollados que están relacionados con las pantallas, con las cuales interactúa el usuario final. En este caso, simplemente se tiene como hijos de la carpeta “views” a “widgets” y “plan_screen.dart”. La carpeta “widgets” contendrá los widgets desarrollados susceptibles a ser empleados en distintas pantallas. Por ahora, solo se tiene la pantalla “plan_screen.dart”, pero en lo posterior se tendrá otras pantallas.

Ahora, se procederá a hablar sobre el código de la aplicación. El archivo principal “main.dart” deberá incorporar la importación de la biblioteca de material, la importación del archivo que contiene el widget *PlanScreen*, el programa principal y el widget *MaterialApp* indicando la pantalla que se renderizará mediante la propiedad *home*, que en este caso será *PlanScreen()*.

```
main.dart

import 'package:flutter/material.dart';
import './views/plan_screen.dart';
void main() => runApp(const PlansApp());
class PlansApp extends StatelessWidget {
  const PlansApp({super.key});
  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: PlanScreen()
    );
  }
}
```

La pantalla *PlanScreen()* es un widget del tipo *StatelessWidget*, que se recordará como un widget sin estado interno, es decir, no puede alterar la información que tiene definida de inicio por si sola.

Su estructura básica es la siguiente:

views/plan_screen.dart

```
import 'package:flutter/material.dart';
class PlanScreen extends StatelessWidget {
  const PlanScreen({super.key});
  @override
  Widget build(BuildContext context) {
    ...
  }
}
```

Se debe importar la biblioteca *material.dart*, posteriormente, se define una clase que debe heredar de *StatelessWidget*. Este tipo de clase debe sobre escribir siempre el método *build*, es ahí donde se define la estructura y contenido de lo que contendrá la pantalla.

A continuación, se muestra el contenido del método *build*.

views/plan_screen.dart

```
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Plan maestro'),
    ),
    body: _buildList(),
    floatingActionButton: const FloatingActionButton(
      onPressed: null,
      tooltip: 'Agregar Plan',
      child: Icon(Icons.add),
    ),
  );
}
```

Este método construye un widget *Scaffold*, en este caso, simplemente presenta tres elementos: un *appBar* con el título “Plan maestro”, el *body* que contendrá una lista desarrollada en el método *_buildList()* y tendrá también un botón *floatingActionButton* planificado para agregar nuevos planes, sin embargo, en

este momento la propiedad *onPressed* indicará *null*, lo que significa que aún no se tiene la implementación para la funcionalidad de agregar nuevos planes.

Se procederá a revisar la construcción de la lista de planes.

views/plan_screen.dart

```
Widget _buildList() {  
  return ListView(  
    children: <Widget>[  
      for (int i = 0; i < 5; i++) ...[  
        Container(  
          margin: const EdgeInsets.all(8.0),  
          padding: const EdgeInsets.all(8.0),  
          decoration: const BoxDecoration(  
            border: Border(  
              bottom: BorderSide(  
                color: Colors.black26,  
                width: 1.0,  
              ),  
            ),  
          ),  
          child: ListTile(  
            title: Text(  
              'Plan ${i + 1}',  
              style:  
                const TextStyle(fontWeight: FontWeight.w500, fontSize: 18.0),  
            ),  
            onTap: () {  
              // Aquí iría tu código para manejar el onTap  
            },  
            trailing: IconButton(  
              icon: const Icon(Icons.delete, color: Colors.redAccent),  
              onPressed: () {  
                // Aquí iría tu código para manejar el onPressed  
              },  
            ),  
          ),  
        ],  
      ],  
    ),  
  );  
}
```

El código define el método `_buildList`, que retorna un widget `ListView`, como ya se explicó, es un contenedor desplazable que alberga una lista de widgets, dentro de este `ListView`, se utiliza una comprensión de lista con un bucle `for` para generar cinco widgets `Container`, cada uno representando un elemento de la lista. Cada `Container` tiene un margen y un relleno uniforme (*margin* y *padding*), así como una decoración que consiste en un borde en la parte inferior, proporcionando una separación visual entre los elementos. Dentro de cada `Container`, se coloca un widget `ListTile` que actúa como el contenido interactivo del elemento de la lista, mostrando un texto titulado '*Plan $\{i + 1\}$* ' donde *i* es el índice del elemento actual en el bucle, comenzando desde 0 hasta 4, resultando en títulos que van desde 'Plan 1' hasta 'Plan 5'. Cada `ListTile` incluye un evento `onTap` vacío, que se supone para manejar interacciones táctiles y un botón `IconButton` con un ícono de eliminación, también con un controlador de eventos `onPressed` vacío, destinado en lo posterior a invocar un diálogo que permita eliminar el elemento de la lista.

2.4.3 Construcción de un diálogo de eliminación

Esta aplicación tendrá dos diálogos, uno para la funcionalidad de agregar planes y otro para hacer la confirmación o cancelación de la eliminación de planes. En este punto, se trabajará en el diálogo que apoyará la eliminación de planes, la razón se da en cuanto a que ya se trató el contenido necesario para lograr este widget. Por otro lado, se retrasará un poco más la construcción del diálogo de agregación, ya que requiere del conocimiento de widgets de entrada, contenido que aún no se tocó, por lo que se implementará en lo posterior. Hecha la aclaración, la imagen muestra el aspecto que tendrá el diálogo de eliminación:

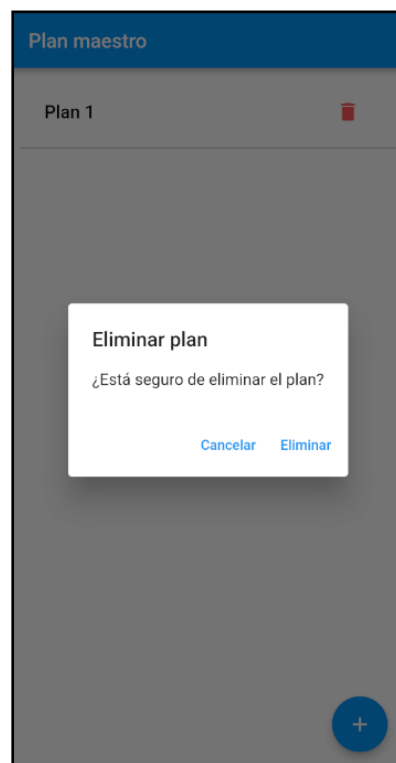


Figura II.14. Diálogo de eliminación

A continuación, se presenta el código que refleja la estructura inicial del widget:

```
views/widgets/delete_dialog.dart

import 'package:flutter/material.dart';
class DeleteDialog extends StatelessWidget {
  final String title;
  final String content;
  final Function() onDelete;
  const DeleteDialog(
    {Key? key,
     required this.title,
     required this.content,
     required this.onDelete})
    : super(key: key);
  @override
  Widget build(BuildContext context) {
    ...
  }
}
```

El código define una clase *DeleteDialog*, que es un widget sin estado (*StatelessWidget*), diseñado para representar un diálogo de confirmación de eliminación. Esta clase toma tres parámetros obligatorios: *title*, *content* y *onDelete*, que son, respectivamente, el título del diálogo, el contenido o mensaje del diálogo y una función callback que se invoca cuando se confirma la acción de eliminación. Estos parámetros son marcados como requeridos mediante la palabra clave *required*, asegurando que proporcionen valores para ellos cada vez que se cree una instancia de *DeleteDialog*. Además, acepta un parámetro opcional *key*, que se proporciona al constructor de la clase base para su uso en el árbol de widgets. La implementación específica del método *build* no se proporciona en este fragmento, pero es donde se definirá la estructura y el comportamiento del diálogo. Ahora, es oportuno presentar ese código.

```
views/widgets/delete_dialog.dart

@override
Widget build(BuildContext context) {
  return AlertDialog(
    title: Text(title),
    content: Text(content),
    actions: <Widget>[
      TextButton(
        onPressed: () => Navigator.of(context).pop(),
        child: const Text("Cancelar"),
      ),
    ],
  );
}
```

```
        TextButton(  
            onPressed: () {  
                onDelete();  
                Navigator.of(context).pop();  
            },  
            child: const Text("Eliminar"),  
        ),  
    ],  
);  
}
```

Este código pertenece al método *build* de la clase *DeleteDialog*, su función es construir y retornar un *AlertDialog*. Este diálogo está diseñado para mostrar una interfaz de usuario que solicita confirmación al usuario antes de ejecutar una acción de eliminación. El diálogo contiene un título y un contenido, que son proporcionados por las variables *title* y *content* respectivamente. Además, incluye dos acciones en forma de botones de texto: uno etiquetado como "Cancelar", que simplemente cierra el diálogo sin realizar ninguna acción adicional y otro etiquetado como "Eliminar". Al presionar el botón "Eliminar", se invoca la función *onDelete* pasada al widget, que se espera que ejecute la lógica de eliminación y luego cierra el diálogo. Esto permite una interacción intuitiva y directa, donde el usuario puede confirmar o cancelar la acción de eliminación propuesta.

Es necesario aclarar, que por el momento no se está añadiendo la funcionalidad para observar el dialogo, por lo que eso queda pendiente para el siguiente capítulo.